



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究
A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於案例式學習與動態計畫修復之 API 任
務大型語言模型代理人自我修正機制研究**
**A Self-Correction Mechanism for LLM Agents in
API Tasks Using Case-Based Learning and Dynamic
Plan Repair**

研究生：陳安琪

指導教授：劉建宏博士

中華民國一百一十五年七月

摘要

關鍵詞：大型語言模型代理人、API 任務自動化、原子化經驗、動態局部修復

大型語言模型代理人能透過應用程式介面 (Application Programming Interface, API) 操作外部系統，但在多步驟任務中，若規劃不完整、資料依賴處理錯誤，或個別步驟執行失敗，後續步驟可能因缺少正確的中間結果而無法完成，進而影響整體任務成果。本研究採用的基礎系統為 API 知識圖譜代理人 (API Knowledge Graph Agent, APIKGAgent)，其主要透過重新規劃處理執行失敗，尚未整合由成功任務軌跡建立的經驗重用機制與局部計畫修復。因此，本研究探討如何運用既有成功經驗，並針對局部錯誤調整計畫，以提升代理人的任務完成能力。本研究以 APIKGAgent 為基礎，提出案例式修復 API 知識圖譜代理人 (Case-Based Repair API Knowledge Graph Agent, CBR-APIKGAgent)，整合原子化經驗 (Atomic Experience) 重用與動態局部修復 (Dynamic Local Repair)。系統從訓練資料的成功任務軌跡中萃取可跨任務重用的策略單元，並依當前任務或失敗情境檢索相關經驗，提供初始規劃、局部修復與重新規劃參考。當步驟執行失敗時，系統根據錯誤訊息、相依資料與執行結果，透過插入、替換或刪除步驟調整局部計畫；若無法形成有效修正，再回退至重新規劃。本研究於 AppWorld 的 168 個多應用程式任務上進行評估。完整系統的任務目標完成率由基礎系統的 37.5% 提升至 51.8%，情境目標完成率則由 14.3% 提升至 35.7%。消融實驗與案例分析顯示，原子化經驗主要協助代理人掌握資料依賴、目標集合與操作條件，動態局部修復則能處理授權缺失、前置資料不足及局部操作不完整等執行錯誤；兩項機制分別作用於規劃與執行階段，結合後取得最佳整體表現。研究亦發現，系統表現仍會受到檢索經驗與任務限制的適配程度、初始規劃品質，以及局部修復與重新規劃判斷等因素影響。整體而言，本研究說明了細粒度經驗重用與局部計畫修復能強化大型語言模型代理人在多步驟 API 任務中的自我修正能力。

Abstract

Keyword: LLM agents, API task automation, atomic experience, dynamic local repair

Large language model agents can operate external systems through application programming interfaces (APIs). In multi-step tasks, however, incomplete plans, incorrectly handled data dependencies, or failed steps may leave subsequent steps without the intermediate results needed for completion, thereby affecting the overall task outcome. The API Knowledge Graph Agent (APIKGAgent) baseline used in this study mainly handles execution failures through replanning and does not integrate experience reuse from successful task trajectories or local plan repair. This thesis therefore proposes the Case-Based Repair API Knowledge Graph Agent (CBR-APIKGAgent), which integrates atomic experience reuse and dynamic local repair. The system extracts transferable strategies from successful training trajectories and retrieves relevant experiences for planning and repair. When a step fails, it uses the failure context to insert, replace, or delete local plan steps, falling back to replanning only when no effective repair can be formed. The system was evaluated on 168 multi-application tasks in AppWorld. Compared with the base system, CBR-APIKGAgent improved task goal completion from 37.5% to 51.8% and scenario goal completion from 14.3% to 35.7%. Ablation experiments and case analysis indicate that atomic experiences mainly support data dependencies, target selection, and operation conditions, while local repair addresses missing authorization, insufficient prerequisite data, and incomplete operations. Combining both mechanisms achieved the best overall performance. System performance nevertheless remains sensitive to the fit between retrieved experiences and task constraints, initial planning quality, and decisions between local repair and replanning. Overall, the results show that fine-grained experience reuse and local plan repair can strengthen self-correction in multi-step API tasks.

誌謝

回顧這段碩士求學歷程，從最初的摸索到最終完成論文，過程中經歷了無數次嘗試與修正。能夠走到這一步，並非憑藉一人之力，而是有賴許多人的陪伴與支持。

首先，誠摯感謝指導教授劉建宏老師。在研究過程中，老師總能在關鍵時刻點出問題核心，引導我重新思考研究方向。無論是研究上的討論，或是面對困難時的提醒，都讓我逐漸建立更成熟的思考方式，也學會以更嚴謹的態度面對學術問題。

同時，也誠摯感謝梁文耀教授的指導與建議。老師在討論中提出的觀點，經常使我跳脫原有框架，讓研究內容更加周全，也使我對自己的研究有了更深入的理解。

在實驗室的日子裡，感謝珮倫與郁喬，在我因論文進度感到沮喪、對自己失去信心時，給予我鼓勵與支持；也感謝冠穎與以謙，在我對研究有所疑惑時，耐心提供幫助與建議。此外，也想感謝一路上關心與支持我的朋友們。在我感到疲憊或迷惘時，是你們的鼓勵讓我得以重新調整步伐，繼續向前。

最後，我想將這份成果獻給我的家人。謝謝你們長久以來的理解與支持，讓我能夠專心投入學業，並在面對挑戰時，始終保有繼續前進的力量。

這段旅程的結束，同時也是另一段道路的開始。謹以此文，向所有曾經幫助與陪伴我的人，致上最真摯的感謝。

目錄

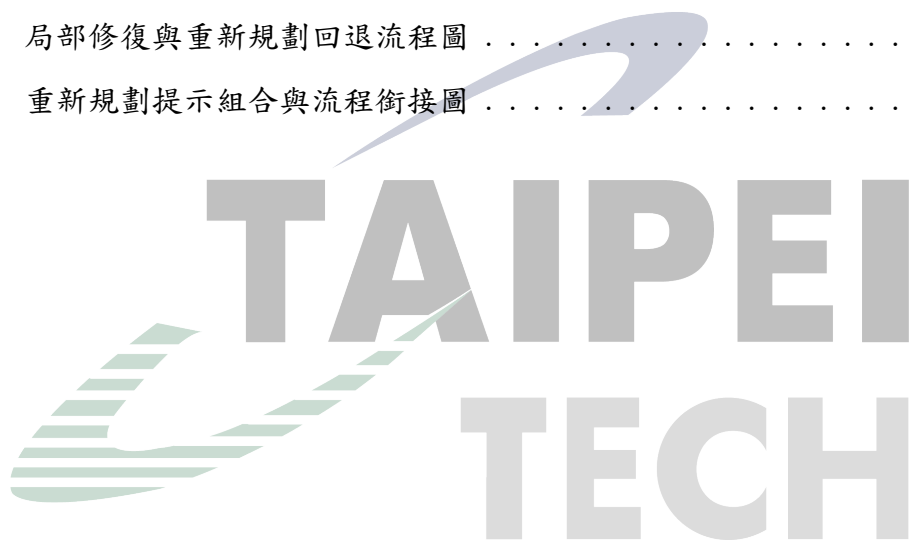
摘要	i
Abstract	ii
誌謝	iii
目錄	iv
圖目錄	vii
表目錄	viii
第一章 緒論	1
1.1 研究動機	1
1.2 研究目的	2
1.3 論文組織架構	2
第二章 相關技術與文獻探討	4
2.1 大型語言模型代理人架構	4
2.2 工具使用與 API 任務自動化	4
2.2.1 多步驟 API 任務特性	4
2.2.2 API 任務自動化相關研究	5
2.2.3 AppWorld 評估環境	5
2.3 案例式學習與經驗重用	6
2.3.1 案例式推理	6
2.3.2 經驗重用相關研究	6
2.3.3 語意檢索方法	7
2.4 任務規劃、動態計畫修復與自我修正	7
2.4.1 任務規劃與重新規劃	7
2.4.2 自我修正機制	8
2.4.3 動態計畫修復相關研究	9
2.5 APIKGAgent	9
第三章 CBR-APIKGAgent 系統架構與方法	11
3.1 CBR-APIKGAgent 系統總覽	11

3.1.1	系統架構與核心機制	11
3.1.2	整體任務執行演算法	13
3.2	原子化經驗重用機制	15
3.2.1	經驗來源與使用範圍	16
3.2.2	原子化經驗定義	17
3.2.3	經驗檢索視角與使用情境	19
3.3	動態局部修復機制	20
3.3.1	局部修復設計目標與適用邊界	20
3.3.2	局部修復與重新規劃之選擇原則	21
3.3.3	局部計畫修復策略	22
第四章	CBR-APIKGAgent 系統實作	25
4.1	原子化經驗庫建構實作	25
4.1.1	訓練資料成功任務軌跡萃取	25
4.1.2	原子化經驗資料格式	26
4.1.3	經驗向量化與雙索引建構	27
4.2	初始規劃實作	28
4.2.1	規劃導向經驗檢索	29
4.2.2	初始計畫生成與提示注入	29
4.3	步驟執行與任務狀態管理	30
4.3.1	執行結果與相依資料保存	31
4.4	失敗處理與局部修復實作	32
4.4.1	錯誤分類與可行回饋產生	33
4.4.2	局部修復觸發與限制檢查	34
4.4.3	修復 API 搜尋與候選整理	35
4.4.4	修復導向經驗注入	35
4.4.5	插入、替換與刪除策略實作	36
4.5	重新規劃實作	37
4.5.1	重新規劃觸發條件	38
4.5.2	重新規劃提示組合與經驗注入	38

4.5.3	重新規劃後的流程銜接	39
第五章	實驗與結果分析	40
5.1	研究問題	40
5.2	實驗設置	41
5.2.1	資料集與任務範圍	41
5.2.2	評估指標	41
5.2.3	實驗環境與主要控制設定	42
5.3	實驗一：整體效能比較	42
5.4	實驗二：消融實驗與案例分析	43
5.4.1	實驗組別	44
5.4.2	消融實驗結果	44
5.4.3	單一機制效益分析	45
5.4.4	完整系統與代表性案例分析	47
第六章	結論與未來研究方向	50
6.1	結論	50
6.2	未來研究方向	51
參考文獻		52
附錄 A	核心提示詞、輸入資料與輸出格式	55
A.1	原子化經驗萃取 Prompt 與輸出範例	55
A.2	錯誤分類與可行回饋 Prompt	58
A.3	局部修復策略選擇 Prompt	59
A.4	規劃與重新規劃的經驗注入片段	61

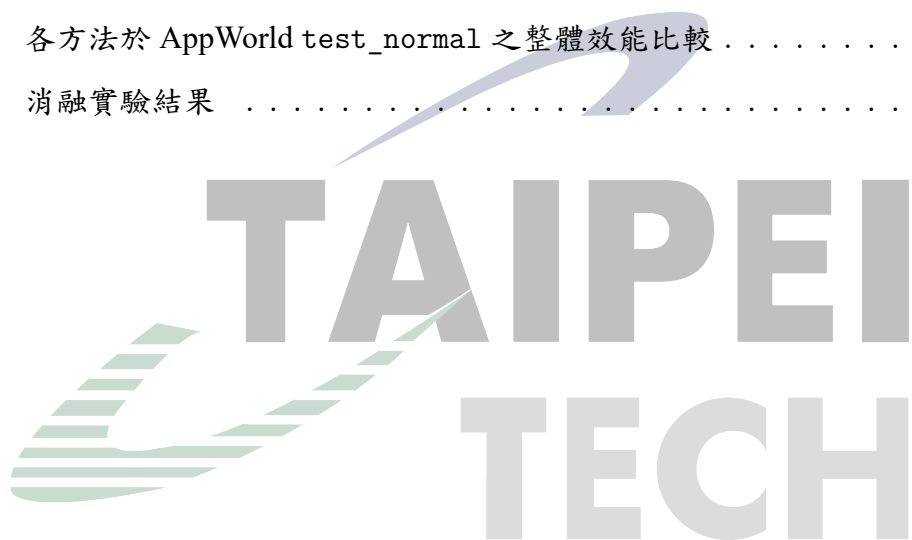
圖目錄

圖 2.1	APIKGAgent 系統架構圖	9
圖 3.1	CBR-APIKGAgent 系統架構圖	11
圖 3.2	原子化經驗萃取與重用流程	16
圖 4.1	初始規劃與經驗注入流程圖	28
圖 4.2	步驟執行與狀態更新流程圖	30
圖 4.3	局部修復與重新規劃回退流程圖	32
圖 4.4	重新規劃提示組合與流程銜接圖	37



表目錄

表 3.1	CBR-APIKGAgent 兩項核心設計之架構角色	12
表 3.2	原子化經驗概念組成	18
表 4.1	錯誤處理與局部修復相關狀態資訊	33
表 5.1	硬體與軟體環境	42
表 5.2	實驗主要控制設定	43
表 5.3	各方法於 AppWorld test_normal 之整體效能比較	44
表 5.4	消融實驗結果	45



第一章 緒論

1.1 研究動機

近年來，大型語言模型（Large Language Model, LLM）已逐漸由傳統的文字生成與問答系統，發展為能夠自主規劃、呼叫工具並操作外部系統的大型語言模型代理人（Large Language Model Agent, LLM Agent）[1]。透過應用程式介面（Application Programming Interface, API），代理人得以執行資料查詢、訊息傳送、檔案管理、行事曆管理及交易處理等實際工作，使大型語言模型由單純提供資訊的輔助工具，進一步成為能與數位環境互動並完成使用者目標的自動化系統。因此，如何提升代理人在真實環境中的任務完成能力與執行可靠性，已成為大型語言模型研究的重要方向之一。

然而，真實世界中的 API 任務通常涉及多個相互依賴的執行步驟，而非單次 API 呼叫即可完成。代理人除了需要理解使用者需求並選擇適當的 API 外，還必須進行多步驟規劃、維護步驟間的資料依賴關係、保存中間執行結果，並持續遵循任務條件與限制。當任務規模增加或操作流程變得複雜時，任何一個步驟的錯誤都可能影響後續決策，使錯誤沿著執行流程持續傳播，最終導致整體任務失敗。其中，可由錯誤訊息與執行結果觀察到的步驟失敗，能提供代理人調整執行流程的依據。因此，如何針對此類可觀察的局部失敗進行適當處理，並降低其對既有計畫與中間資料的影響，是提升多步驟 API 代理人可靠性的重要挑戰。

基於上述多步驟 API 任務的特性，以及本研究採用之 API 知識圖譜代理人（API Knowledge Graph Agent, APIKGAgent）[2] 基礎流程，本研究聚焦於下列兩項問題：

- **全域經驗利用不足：**不同任務可能具有相似的規劃與資料依賴問題，例如目標集合辨識、必要中間資料取得、分頁資料完整性確認，以及寫入前條件檢查等。APIKGAgent 的基礎流程尚未將訓練資料中成功任務軌跡的策略知識整理為可檢索、可重用的經驗，因此在面對相似任務結構時，仍需依賴當下推理形成規劃與修復依據。
- **局部錯誤修復能力不足：**APIKGAgent 的基礎流程主要透過重新規劃處理任務執行錯誤。然而，部分失敗可能僅與當前步驟附近的前置資料缺失、API 選擇錯誤或參數設定不當有關。若針對此類局部失敗仍重新產生較大範圍的計畫，可能造成不必要的整體變動，也可能干擾原本仍然有效的執行流程與中間資料。

基於上述觀察，本研究認為提升多步驟 API 代理人可靠性的關鍵，不僅在於大型語言模型本身的推理能力，也在於如何利用由訓練資料成功任務軌跡所萃取的策略知識，並針對執行期間的局部錯誤進行適當修復。因此，本研究探討如何從訓練資料中的成功任務軌跡萃取細粒度經驗，在正式任務執行前建立可供不同任務共用的經驗庫，並透過語意檢索將相關經驗應用於初始規劃、重新規劃與局部修復流程；同時研究如何根據執行結果、步驟相依資料與錯誤證據進行動態局部修復。

為具體評估上述機制，本研究選擇 APIKGAgent 作為研究基礎。APIKGAgent 透過 API 知識圖譜、API 搜尋、任務規劃、步驟式執行與重新規劃機制，提供多步驟 API 任務代理人的基本流程。基於此流程，本研究進一步探討原子化經驗（Atomic Experience）重用與動態局部修復（Dynamic Local Repair）是否能為相似任務結構提供規劃與修復參考，並協助代理人由執行失敗中恢復，進而評估兩項機制對多步驟 API 任務完成能力與錯誤修復表現的影響。

1.2 研究目的

本研究以 APIKGAgent 為基礎，提出案例式修復 API 知識圖譜代理人（Case-Based Repair API Knowledge Graph Agent, CBR-APIKGAgent），整合原子化經驗重用與動態局部修復機制，以探討其對多步驟 API 任務完成與失敗恢復能力的影響。

具體而言，本研究之目的如下：

1. 建立訓練資料成功經驗的萃取與重用機制

本研究從訓練資料中的成功任務軌跡萃取可跨任務重用的原子化經驗，並在規劃與修復階段提供相關策略參考。

2. 建立動態局部修復機制以處理執行失敗

本研究根據執行失敗的錯誤訊息與相依資料調整局部計畫，並在無法形成有效修正時回退至重新規劃。

3. 評估兩項機制對任務完成的影響

本研究透過 AppWorld [3] 實驗、消融分析與案例分析，評估原子化經驗與動態局部修復的個別作用、共同影響及限制。

1.3 論文組織架構

本論文共分為六章，各章內容概述如下：

- **第一章緒論：**說明多步驟 API 任務的研究背景與動機、APIKGAgent 基礎流程所面臨的問題、本研究目的，以及論文組織架構。
- **第二章相關技術與文獻探討：**整理大型語言模型代理人、API 任務自動化、案例式學習與經驗重用、語意檢索、動態計畫修復與自我修正，以及 APIKGAgent 等相關技術與研究。
- **第三章 CBR-APIKGAgent 系統架構與方法：**介紹 CBR-APIKGAgent 的系統總覽與整體任務執行流程，並說明原子化經驗重用與動態局部修復兩項核心機制。

- **第四章 CBR-APIKAgent 系統實作：**說明原子化經驗庫建構、初始規劃、步驟執行與任務狀態管理、失敗處理與局部修復，以及重新規劃等實作流程。
- **第五章實驗與結果分析：**說明研究問題、實驗設置與評估指標，並透過整體效能比較、消融實驗與代表性案例分析，評估原子化經驗與動態局部修復的個別作用、共同影響及限制。
- **第六章結論與未來研究方向：**總結研究結果，並說明本研究目前的限制與後續可延伸之研究方向。

此外，附錄 A 收錄原子化經驗萃取、錯誤分類與可行回饋、局部修復策略選擇，以及規劃與重新規劃經驗注入所使用的核心提示詞與輸出範例。



第二章 相關技術與文獻探討

本章依研究問題所涉及的技術面向整理相關研究。首先介紹大型語言模型代理人的基本架構，以及工具使用與 API 任務自動化的研究背景；接著說明案例式學習、經驗重用與語意檢索方法；再探討任務規劃、重新規劃、自我修正與動態計畫修復；最後介紹本研究所延伸之 APIKGAgent，並綜合比較上述研究與本研究欲處理的問題。

2.1 大型語言模型代理人架構

大型語言模型代理人（Large Language Model Agent, LLM Agent）係指以大型語言模型（Large Language Model, LLM）作為決策核心，並結合記憶、外部工具與環境互動能力以完成任務的代理系統。相較於僅產生文字回應的大型語言模型，此類代理人會根據任務目標、歷史資訊與環境狀態形成決策，再透過工具呼叫或其他行動與外部系統互動。

Wang 等人 [1] 提出大型語言模型自主代理人的統一架構，將代理人分為角色設定（Profiling）、記憶（Memory）、規劃（Planning）與行動（Action）四個核心模組。角色設定模組定義代理人的角色與行為特徵；記憶模組保存互動資訊，作為後續決策依據；規劃模組將複雜任務拆解為可執行的子任務；行動模組則將決策轉化為工具呼叫或其他具體輸出。此架構呈現代理人從保存任務脈絡、形成計畫到執行外部行動的基本組成。

2.2 工具使用與 API 任務自動化

由於大型語言模型本身無法直接存取外部系統，因此工具使用（Tool Use）已成為現代 LLM Agent 的核心能力之一。工具使用係指代理人透過 API、函式呼叫（Function Calling）或其他外部工具介面執行特定操作，例如資料查詢、網頁搜尋、資料庫存取或檔案處理等。API 任務自動化（API Task Automation）則以此能力為基礎，使代理人能依據任務需求選擇與呼叫 API，並利用執行結果完成單一步驟或多步驟操作。

Schick 等人提出 Toolformer，使大型語言模型學習何時呼叫工具、如何選擇工具與填入參數，並將工具回傳結果納入後續生成 [4]。此類工具使用能力使代理人得以透過外部介面取得資訊或改變環境狀態，也構成 API 任務自動化的技術基礎。

2.2.1 多步驟 API 任務特性

API 任務係指代理人透過呼叫一個或多個 API 查詢或改變外部環境，以完成特定目標的過程。與一般問答任務不同，此類任務需要實際呼叫外部介面，並依工具回傳結果或環境狀態判斷任務是否完成。

簡單任務可能僅需單次呼叫，複雜任務則通常包含多個相互依賴的步驟。例如，代理人可能需要先搜尋符合條件的使用者，再取得詳細資訊，最後執行修改或刪除操作。此類任務涉及工具選擇、參數建構、執行順序與資料依賴管理，且具有狀態性 (Statefulness)：前一步驟的結果可能成為後續操作的條件，因此必須正確保存與使用中間資料。若其中一步發生錯誤，其影響可能延伸至後續流程，最終導致任務失敗。

2.2.2 API 任務自動化相關研究

在大規模 API 環境中，不同研究分別從模型訓練、文件檢索與階層式工具選擇提升 API 呼叫能力。Patil 等人提出 Gorilla，結合 API 文件檢索與檢索器感知訓練 (Retriever-Aware Training, RAT)，使模型能根據使用者指令產生 API 呼叫，並適應測試階段文件所反映的 API 版本或參數變化 [5]。Qin 等人提出 ToolLLM，利用大規模工具使用資料、呼叫軌跡與檢索機制，使模型能從大量候選工具中選擇功能並產生所需參數 [6]。Du 等人提出 AnyTool，由階層式 API 檢索器、任務求解器與自我反思機制組成，並利用 GPT-4 的函式呼叫能力處理大規模 API 選擇與使用，無須另行訓練外部模組 [7]。上述方法顯示，文件檢索、模型訓練與階層式搜尋皆可作為大規模 API 任務自動化的實現途徑。

除了從大量候選工具中產生 API 呼叫外，API 任務自動化亦涉及完整的任務規劃與執行。Song 等人提出 RestGPT，將大型語言模型連接至真實的 RESTful API，透過由粗至細的線上規劃機制進行任務分解與 API 選擇，並由專用執行器建構參數及解析 API 回應 [8]。此方法將 API 選擇、參數生成與回傳資料處理整合為多步驟自動化流程，呈現代理人依據 API 文件與執行結果逐步完成複雜指令的方式。

2.2.3 AppWorld 評估環境

AppWorld 由可控制的應用程式執行環境 AppWorld Engine 與任務基準 AppWorld Benchmark 組成，用於評估大型語言模型代理人執行日常多步驟數位任務的能力 [3]。其執行環境包含 9 個日常應用程式、457 個 API，以及模擬約 100 位虛構使用者之數位活動資料；任務基準則包含 750 個自然語言任務，要求代理人根據中間執行結果，在一個或多個應用程式中完成查詢、資料處理與狀態變更。此外，AppWorld 提供以 IPython 為基礎的具狀態執行殼層 (Stateful Execution Shell)，使代理人能以互動方式撰寫與執行 Python 程式碼、檢視執行結果，並重用先程式區塊所建立的變數。

AppWorld 透過以最終環境狀態為基礎的程式化測試判斷任務是否完成，因此不要求代理人遵循單一固定解答路徑，並能檢查任務目標以外的非預期狀態變更，即附帶損害 (Collateral Damage)。相較於主要評估單次工具選擇或短序列 API 呼叫的基準，AppWorld 任務平均涉及 1.8 個應用程式與 9.5 次 API 呼叫，包含較長的操作流程與相互依賴的中間資料，可用於觀察代理人的任務規劃、工具呼叫、資料處理與動態調整能力 [3]。

2.3 案例式學習與經驗重用

任務經驗可透過案例表示、外部記憶與語意檢索支援代理人的後續決策。本節依序介紹案例式推理、代理人經驗重用相關研究與語意檢索方法，說明案例知識如何被表示、保存與取用。

2.3.1 案例式推理

案例式推理 (Case-Based Reasoning) 是一種以過往問題求解經驗處理新問題的方法。Aamodt 與 Plaza 將其概括為檢索 (Retrieve)、重用 (Reuse)、修訂 (Revise) 與保留 (Retain) 四個階段所構成的循環 [9]。檢索階段從案例庫中找出與當前問題相似的案例；重用階段將既有案例的解法套用或調整至新問題；修訂階段根據實際結果檢查並修正解法；保留階段則將新的求解經驗整理後存回案例庫，供後續問題使用。

案例式推理中的案例通常由問題描述、解決方案及結果等資訊組成，而案例是否有效重用，取決於情境表示、相似性判斷與解法調整方式。若案例表示過於具體，檢索結果可能只適用於幾乎相同的任務；若抽象程度過高，則可能失去執行時所需的條件與限制。因此，案例設計需在可重用性與情境完整性之間取得平衡。

Wilkerson 與 Leake 探討以完整案例作為大型語言模型的外部資訊，並引導模型執行案例式推理部分流程的可行性 [10]。其實驗顯示案例可改善特定分類任務的表現，但模型在案例相似性判斷上的能力有限，因此支持將案例選擇交由獨立檢索機制處理。該研究提供案例式推理與大型語言模型結合的近期依據，但未直接處理多步驟 API 計畫修復。

2.3.2 經驗重用相關研究

大型語言模型代理人的經驗可依來源概分為兩類：第一類是在互動或任務嘗試期間形成的觀察與反思記憶，主要支援代理人延續互動或改善後續嘗試；第二類則是從既有任務資料整理出的跨任務經驗，用於協助代理人處理新的相似任務。

在互動與任務嘗試期間形成經驗的研究中，Park 等人提出 Generative Agents，將代理人的觀察保存於記憶串流，並根據時近性、重要性與相關性檢索記憶，同時透過反思產生較高層次的概念 [11]。Reflexion 則根據任務回饋產生文字反思並保存於記憶中，使代理人在不更新模型參數的情況下，利用先前嘗試的結果調整後續決策 [12]。這類方法著重代理人在互動歷程或同一任務的多次嘗試中累積可供後續使用的資訊。

另一類研究著重從多個既有任務中整理可跨任務重用的經驗。Zhao 等人提出 ExpeL，從訓練任務軌跡萃取自自然語言洞察，並於推論階段檢索相似的成功軌跡，共同輔助新任務決策 [13]。Wang 等人提出 Agent Workflow Memory (AWM)，從過往任務軌跡中歸納可重複使用的工作流程，並在後續任務中選擇性提供相關流程以引導行動生成 [14]。AWM 可離線從訓練案例建立工作流程，也可在線上執行期間持續歸納經驗。這些研究顯示，任務軌跡可被整理為不同粒度的外部知識，並支援跨任務決策。

2.3.3 語意檢索方法

經驗庫規模增加後，系統需要從大量經驗中選出與當前情境相關的內容。傳統關鍵字檢索主要依賴詞彙重疊，當任務與經驗使用不同描述方式時，可能無法找出語意相近的案例。語意檢索（Semantic Retrieval）則將查詢與候選內容轉換為稠密向量，並在向量空間中計算相似程度。Karpukhin 等人提出 Dense Passage Retrieval，利用雙編碼器分別建立查詢與文本的稠密表示，說明稠密向量可用於找出語意相關而不僅是詞彙相同的內容 [15]。

Lewis 等人提出檢索增強生成（Retrieval-Augmented Generation, RAG），將預訓練生成模型視為參數化記憶，並以可檢索的稠密向量索引作為非參數化記憶，使模型能依據查詢取得外部內容後再進行生成 [16]。此概念使知識可獨立於模型參數保存與更新，也適合用於將任務經驗提供給大型語言模型代理人參考。

語意檢索常使用餘弦相似度衡量查詢向量 $\mathbf{q}$ 與經驗向量 $\mathbf{e}$ 的接近程度，其定義如下：

$$\text{sim}(\mathbf{q}, \mathbf{e}) = \frac{\mathbf{q} \cdot \mathbf{e}}{\|\mathbf{q}\| \|\mathbf{e}\|}. \quad (2.1)$$

系統可依相似度排序並取得前 k 筆經驗，再將其加入代理人的提示內容。然而，語意相似並不代表策略必然適用；若查詢內容未包含當前階段的目標、失敗資訊或資料依賴，仍可能檢索到表面相似但無法使用的經驗。此外，檢索數量過多也可能增加無關資訊，干擾模型原有判斷。因此，語意相似度較適合作為候選經驗的初步篩選依據，而非判定經驗是否適用的唯一標準；經驗內容的結構、查詢資訊的選擇與檢索結果的注入方式，皆會影響經驗重用的實際表現。

2.4 任務規劃、動態計畫修復與自我修正

大型語言模型代理人在多步驟任務中通常需要先將目標拆解為可執行步驟，再依序呼叫工具或 API 完成任務。若執行結果不符合預期，代理人還需利用環境回饋與錯誤資訊，透過重新規劃、自我修正或計畫修復調整後續行動。本節依序說明任務規劃與重新規劃、自我修正，以及動態計畫修復相關研究。

2.4.1 任務規劃與重新規劃

任務規劃（Task Planning）是指根據任務目標將複雜問題拆解為較小的子任務，並安排其執行順序與相依關係。Wang 等人提出 Plan-and-Solve 提示方法，先制定計畫將問題拆分為子任務，再依計畫逐步求解，以減少多步驟推理中的遺漏步驟問題 [17]。該研究主要探討數學、常識與符號推理，未直接處理代理人呼叫外部工具時的執行失敗，但說明先規劃再執行有助於組織多步驟推理流程。

重新規劃 (Replanning) 是多步驟代理人常見的錯誤處理方法。當代理人在執行過程中遇到錯誤或原始計畫無法繼續完成時，系統會根據目前狀態、已完成步驟、失敗資訊與任務目標重新產生後續計畫。此機制使代理人能夠在初始計畫不完整或環境狀態發生變化時，重新調整任務執行方向。

除了明確重新產生任務計畫的機制外，部分代理人方法亦會根據工具執行回饋動態調整後續決策。例如，ReAct 將推理與行動交替進行，使模型能夠根據工具回傳結果更新下一步推理與行動 [18]。ReAct 本身並非重新產生完整任務計畫的方法，但其依據執行觀察調整後續決策的設計，呈現了執行回饋在代理人決策修正中的作用。

然而，對於多步驟 API 任務而言，重新規劃通常涉及重新產生後續計畫，修正範圍相對較大。部分系統雖可辨識已完成步驟並沿用其結果，但新舊計畫在步驟結構或資料依賴上仍可能存在差異。因此，當失敗僅涉及前置資料缺失、參數錯誤或單一 API 選擇不當時，保留既有計畫並針對失敗區段進行調整，可作為另一種較小範圍的處理方式。

2.4.2 自我修正機制

自我修正 (Self-Correction) 是指大型語言模型或代理人根據自身輸出、外部回饋或執行結果，對原始答案、推理過程或行動計畫進行檢查與修改的能力。此類方法通常不需要更新模型參數，而是透過提示設計、回饋訊號或記憶機制，使模型在推論階段改善原始輸出。依回饋取得與使用的時機不同，修正可能發生於單次輸出的反覆生成過程，也可能在一次任務嘗試結束後，將其結果用於後續嘗試。

Reflexion 提出語言強化學習 (Verbal Reinforcement Learning) 架構，使代理人在一次任務嘗試取得回饋後產生文字反思，並將其作為後續嘗試的決策依據 [12]。文字形式的失敗分析與經驗描述因而可在不進行模型微調的情況下，輔助代理人改善後續行動。

Self-Refine 使大型語言模型透過自我回饋與反覆修訂流程改善初始輸出 [19]。其核心概念為先產生結果，再由模型自行分析缺失並提出修改建議，最後依據回饋重新生成內容，使模型在不進行額外訓練的情況下仍能逐步修正輸出。

Zhang 等人提出 Divide-Verify-Refine (DVR)，將複雜指令拆分為個別限制，利用工具驗證輸出並依文字回饋進行修正，同時重用成功修正案例，以結合外部驗證與案例支援輸出修正 [20]。

Zhu 等人提出 AgentErrorTaxonomy，將代理人失敗歸納為記憶、反思、規劃、行動與系統等面向，並以 AgentDebug 定位錯誤及產生修正回饋 [21]。結構化錯誤描述可作為後續修正的依據，但錯誤分類本身仍不必然等同於完整的根因診斷。

在工具使用與 API 任務場景中，自我修正的回饋來源不僅包含模型自身判斷，也包含 API 回傳結果、錯誤訊息、執行紀錄與中間資料狀態。這些外部訊號可協助代理人判斷失敗位置與可能原因，進而針對當前步驟進行調整，而非完全依賴重新產生整體計畫。

2.4.3 動態計畫修復相關研究

動態計畫修復（Dynamic Plan Repair）關注代理人在執行過程中面對錯誤、環境變化或計畫不可行時，如何根據當前狀態修正原有計畫。相較於重新產生完整計畫，計畫修復更強調保留原計畫中仍然有效的部分，並針對失敗區段進行局部調整。

Fox 等人以計畫穩定性（plan stability）比較重新規劃與計畫修復，探討因應新情境時保留原計畫內容及降低計畫變動的重要性 [22]。針對大型語言模型代理人，Sun 等人提出 AdaPlanner，以閉環方式根據計畫內與計畫外回饋調整執行決策或修訂計畫，並可從中間狀態繼續執行 [23]。Prasad 等人則提出 ADaPT，在執行器無法完成特定子任務時，依需要進一步分解該子任務，使代理人能針對失敗部分調整分解粒度，而不必立即放棄整體任務 [24]。

綜合而言，上述方法處理失敗的方式與範圍各不相同，包括修正模型輸出、分析錯誤原因、調整子任務，以及修改既有計畫。

2.5 APIKGAgent

API 知識圖譜代理人（API Knowledge Graph Agent, APIKGAgent）為吳柏諭所提出之大型語言模型代理人架構，其核心概念為結合 API 知識圖譜（API Knowledge Graph）與大型語言模型，使代理人能夠在大量 API 環境中完成多步驟任務 [2]。相較於直接將完整 API 文件提供給大型語言模型進行推理，APIKGAgent 透過知識圖譜保存 API、參數以及參數依賴關係，並利用圖譜搜尋結果輔助任務規劃與執行，以降低大型語言模型於 API 檢索與參數推理過程中的錯誤。其系統架構如圖 2.1 所示。

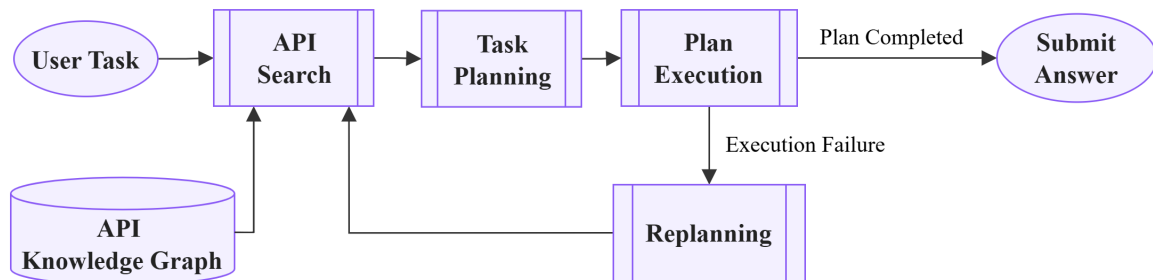


圖 2.1: APIKGAgent 系統架構圖（改繪自 [2]）

APIKGAgent 的整體流程可分為 API 搜尋、任務規劃、任務執行與重新規劃四個階段，其主要內容可整理如下：

- **API 搜尋：**系統根據使用者任務描述擷取關鍵資訊，並透過 API 知識圖譜搜尋可能相關之 API。由於知識圖譜保存了 API 與參數之間的依賴關係，系統除了能縮小初始搜尋範圍，也能在執行過程缺少必要參數時，追溯可提供該參數的 API，作為後續規劃或重新規劃的候選來源。

- **任務規劃：**完成 API 搜尋後，任務規劃模組會根據使用者需求與候選 API 集合產生任務計畫（Task Plan）。系統將複雜任務拆解為多個步驟，並決定各步驟所需之 API 呼叫順序與資料流向，使後續執行流程能夠依照規劃逐步完成任務。
- **任務執行：**在任務執行階段，代理人依照規劃結果逐步執行 API 呼叫與資料處理流程。任務計畫中的步驟可包含 API 呼叫、資料處理、條件跳躍與終止步驟等類型，使系統能區分外部操作、中間資料整理、執行路徑調整與任務結束條件。執行過程中所產生的中間結果會被保存，供後續步驟使用，以維持多步驟任務中的資料依賴關係。此外，系統亦會記錄各步驟之執行結果，作為後續錯誤分析與重新規劃的依據。
- **重新規劃：**當任務執行失敗或無法繼續完成時，系統則啟動重新規劃機制。重新規劃模組會根據目前執行狀態、錯誤資訊以及已完成步驟重新產生後續計畫。新計畫產生後，Pass 機制會比對歷史執行紀錄並重用其中等效且仍有效的成功結果，以減少重複呼叫或非預期的重複操作。

綜合上述研究，工具使用與 API 任務自動化相關方法著重工具選擇、參數生成及多步驟操作，使代理人得以與外部系統互動；案例式推理與經驗重用方法則將過往案例、反思或任務軌跡保存為外部知識，支援後續任務決策。任務規劃與自我修正研究進一步處理步驟拆解、執行回饋與輸出修訂，而動態計畫修復則強調在環境變化或步驟失敗時保留仍有效的計畫內容。這些研究分別涵蓋工具操作、經驗利用與錯誤處理，但所使用的經驗來源、表示粒度及修正範圍並不相同。

APIKGAgent 已能利用知識圖譜協助 API 與參數搜尋，並在重新規劃後重用仍有效的執行結果。然而，其基礎架構尚未將成功任務軌跡整理為可跨任務檢索的細粒度策略，也未提供直接插入、替換或刪除局部計畫步驟的修復方式。因此，本研究聚焦於兩項研究缺口：如何從既有成功任務中建立可重用的原子化經驗，以及如何在執行失敗後優先利用錯誤脈絡調整局部計畫，並於局部修復不適用時再回退至重新規劃。

第三章 CBR-APIKGAgent 系統架構與方法

本章說明 CBR-APIKGAgent 的系統架構與核心方法。此架構延伸 APIKGAgent 的多步驟 API 任務流程，加入原子化經驗重用與動態局部修復兩項機制：前者從訓練資料的成功任務軌跡萃取可轉移的策略知識，後者在步驟執行失敗後優先調整失敗步驟附近的計畫。本文將由既有成功任務取得策略知識並用於新任務的方式稱為案例式學習（Case-Based Learning）。以下先介紹整體架構與任務流程，再分別說明原子化經驗的萃取與使用方式，以及局部修復的計畫調整方法。

3.1 CBR-APIKGAgent 系統總覽

本節先說明原子化經驗重用與動態局部修復在整體架構中的角色，再以演算法呈現 CBR-APIKGAgent 的完整任務執行流程。

3.1.1 系統架構與核心機制

為說明原子化經驗重用與動態局部修復在基本任務流程中的位置，CBR-APIKGAgent 的整體架構如圖 3.1 所示。

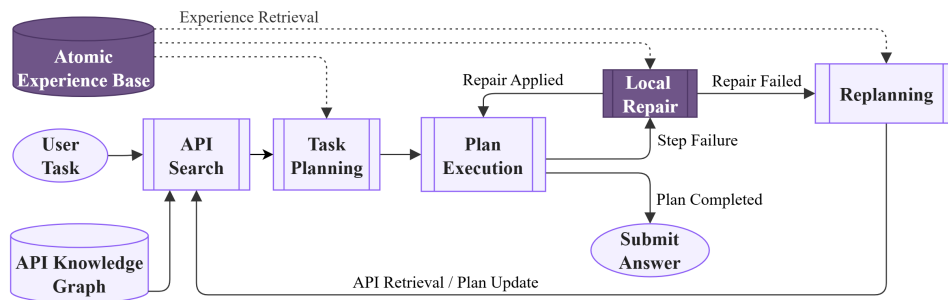


圖 3.1: CBR-APIKGAgent 系統架構圖

圖中淺紫色元件表示沿用 APIKGAgent 的基本任務流程與相關元件，包括使用者任務、API 知識圖譜、API 搜尋、任務規劃、計畫執行、重新規劃與結果提交；深紫色元件則表示本研究新增的原子化經驗庫與局部修復模組。實線箭頭表示任務執行、控

制轉移與計畫更新流程，虛線箭頭則表示原子化經驗由經驗庫檢索至任務規劃、局部修復與重新規劃三個決策階段。

CBR-APIKGAgent 在多步驟 API 代理人架構中加入原子化經驗重用與動態局部修復兩項核心機制。前者提供規劃與修復時的案例式策略參考，後者在步驟失敗後嘗試以局部計畫修改恢復執行。此設計使代理人在面對相似任務結構或局部執行失敗時，能取得來自訓練資料成功任務軌跡的策略參考，而不僅依賴當下錯誤訊息重新產生後續動作。

在整體架構中，API 搜尋、任務規劃與任務執行構成基本任務流程；原子化經驗檢索（Atomic Experience Retrieval）支援初始規劃、重新規劃與局部修復三個決策點；局部修復模組則在步驟失敗後嘗試小範圍計畫修改，並於無法形成有效修改時交由重新規劃處理。案例式經驗檢索負責提供與當前任務或錯誤情境相關的策略參考，屬於決策輔助機制；動態局部修復則負責根據失敗資訊修改局部計畫，屬於錯誤處理機制。

系統接收使用者任務後，先透過 API 知識圖譜搜尋候選 API，再由任務規劃模組結合任務需求、候選 API 與規劃導向經驗產生初始計畫。計畫進入執行階段後，若所有步驟順利完成，系統即提交任務結果；若步驟失敗，則將失敗資訊交由局部修復模組處理。局部修復模組會參考修復導向經驗，嘗試修改失敗步驟附近的計畫內容；若修復成功，系統回到更新後的計畫繼續執行，若無法形成有效修正，則回退至重新規劃。重新規劃階段會綜合原始任務、既有計畫、失敗資訊與檢索經驗產生新計畫，再銜接後續搜尋、規劃與執行流程。

為了使後續方法說明能聚焦於本研究提出的核心機制，表 3.1 以兩項設計為主軸，整理原子化經驗重用與動態局部修復機制在整體架構中的角色。

表 3.1: CBR-APIKGAgent 兩項核心設計之架構角色

核心設計	架構角色	核心作用
原子化經驗重用	決策輔助層	由原子化經驗庫與語意檢索流程提供任務結構、資料依賴、常見陷阱與修復策略等參考。
動態局部修復	錯誤處理層	在步驟失敗後整理失敗上下文，結合候選修復 API 與修復經驗嘗試修改局部計畫；若無法形成有效修改，則回退至重新規劃。

在資料流設計上，CBR-APIKGAgent 以任務狀態作為各階段之間共享上下文的載

體。任務狀態保存任務目標、候選 API、目前計畫、已完成步驟結果與失敗資訊，使規劃、執行、局部修復與重新規劃能在同一脈絡下進行。另一方面，原子化經驗庫提供規劃與修復所需的案例式策略參考；其經驗來源與使用範圍將於第 3.2 節說明。較細部的狀態欄位、資料保存方式與程式傳遞流程，則於第四章系統實作中說明。

3.1.2 整體任務執行演算法

本節以演算法摘要呈現 CBR-APIKGAgent 從任務輸入到結果輸出的完整任務執行流程。演算法中與經驗檢索相關的步驟，將於第 3.2 節進一步說明；與步驟失敗、局部修復及重新規劃回退相關的步驟，則於第 3.3 節展開說明。

Algorithm 1: CBR-APIKGAgent Overall Task Execution Algorithm

Input: user task T , API knowledge graph G , atomic experience base $\mathcal{E}$

Output: execution result R

```

1  $C_{api} \leftarrow \text{SearchAPIs}(G, T)$ ;
2  $\mathcal{E}_p \leftarrow \text{RetrievePlanningExperience}(\mathcal{E}, T)$ ;
3  $P \leftarrow \text{GeneratePlan}(T, C_{api}, \mathcal{E}_p)$ ;
4 while  $\text{HasNextStep}(P)$  do
5    $s \leftarrow \text{NextStep}(P)$ ;
6    $o \leftarrow \text{ExecuteStep}(s)$ ;
7   if  $\text{IsSuccess}(o)$  then
8      $\text{UpdateState}(s, o)$ ;
9   else
10     $F \leftarrow \text{CollectFailureContext}(s, o)$ ;
11     $(c, f) \leftarrow \text{AnalyzeFailure}(F)$ ;
12     $P_{\text{repair}} \leftarrow \emptyset$ ;
13    if  $\text{IsRepairAllowed}(c, F)$  then
14       $C_{\text{repair}} \leftarrow \text{SearchRepairAPIs}(G, F, f)$ ;
15       $\mathcal{E}_r \leftarrow \text{RetrieveRepairExperience}(\mathcal{E}, F)$ ;
16       $m \leftarrow \text{SelectRepairMode}(F, C_{\text{repair}}, \mathcal{E}_r)$ ;
17      if  $m \neq \emptyset$  then
18         $P_{\text{repair}} \leftarrow \text{GenerateRepairPlan}(P, s, m)$ ;
19    if  $P_{\text{repair}} \neq \emptyset$  then
20       $P \leftarrow P_{\text{repair}}$ ;
21       $\text{SetNextStepToRepairPoint}(P)$ ;
22    else
23       $\mathcal{E}_{rp} \leftarrow \text{RetrieveReplanningExperience}(\mathcal{E}, T, P, F)$ ;
24       $P \leftarrow \text{Replan}(T, P, F, \mathcal{E}_{rp})$ ;
25 return  $R$ 

```

演算法 1 的詳細步驟說明如下。為了便於對照演算法結構，以下依照初始化階段、主要執行迴圈與結果回傳三個部分說明。

1. **初始化階段：建立初始計畫。**系統接收使用者任務 T 後，先根據 API 知識圖譜 G 與原子化經驗庫 $\mathcal{E}$ 建立可執行的初始計畫 P ，作為後續逐步執行的計畫依據。

- **API 搜尋：**系統以 T 作為查詢目標，於 G 中搜尋與任務相關的 API，形成初始候選 API 集合 C_{api} 。
- **規劃經驗檢索：**系統同時以 T 作為經驗查詢脈絡，從 $\mathcal{E}$ 中取得與任務結構、資料依賴或操作順序相關的規劃導向經驗 $\mathcal{E}_p$ 。
- **初始計畫生成：**任務規劃器將 T 、 C_{api} 與 $\mathcal{E}_p$ 作為輸入，產生初始計畫 P 。此計畫包含 API 呼叫、資料處理與步驟相依關係。

2. **主要執行迴圈：執行步驟並處理失敗。**產生初始計畫 P 後，系統進入主要執行迴圈。每次迭代會從目前計畫 P 中取出待執行步驟 s ，執行後得到步驟執行結果 o ；系統再依 o 判斷是否更新狀態或進入錯誤處理流程。

- **步驟執行：**系統從目前計畫 P 中取出下一個尚未完成的步驟 s ，並依 s 的內容執行 API 呼叫或資料處理動作。若 s 需要使用前面 API 回傳的資料，執行器會根據 P 中的相依關係取得對應中間結果，再產生執行結果 o 。
- **成功狀態更新：**若 o 表示步驟執行成功，系統會將 s 的輸出寫回任務狀態，供後續相依步驟使用。
- **失敗脈絡整理：**若 o 表示步驟執行失敗，系統會將 s 、 o 、錯誤訊息、API 回饋與相關相依資料整理為失敗脈絡 F ，供後續錯誤分類、局部修復與重新規劃使用。
- **錯誤分類與可行回饋：**系統根據 F 產生暫定錯誤分類 c 與可行回饋 f ，前者用於輔助錯誤處理路由，後者用於描述可能採取的修復方向。
- **局部修復嘗試：**若 c 與修復嘗試狀態未觸發直接重新規劃條件，系統會根據 F 與 f 搜尋可能有助於修復的 API，形成修復候選 API 集合 C_{repair} ，並從 $\mathcal{E}$ 中檢索與目前失敗情境相關的修復導向經驗 $\mathcal{E}_r$ 。
- **修復策略選擇：**系統將 F 、 C_{repair} 與 $\mathcal{E}_r$ 作為修復策略判斷的主要脈絡，決定局部修改方式 m ，例如插入、替換或刪除等計畫修改方式。

- **產生修復後計畫：**若 m 不為空，系統會依據局部修改方式 m 對目前計畫 P 進行局部修改，產生修復後計畫 P_{repair} 。此階段只改寫計畫內容，不直接執行新增或替換後的 API 步驟。例如，系統可在缺少前置資料時插入查詢步驟，於原步驟 API 不適用時替換步驟，或在步驟重複時刪除該步驟。
- **設定後續執行位置：**若 P_{repair} 成功產生，系統會以 P_{repair} 更新 P ，並將後續執行位置調整至修復後計畫中需要接續執行的位置。若採用插入或替換策略，下一次取出的步驟會是新插入或替換後的修復步驟；若採用刪除策略，下一次取出的步驟則會是刪除後補位至該位置的後續步驟。
- **重新規劃回退：**若系統未形成 P_{repair} ，則檢索重新規劃相關經驗 $\mathcal{E}_{\text{rp}}$ 。其中， $\mathcal{E}_{\text{rp}}$ 同時包含依任務目標與既有計畫檢索之規劃導向經驗，以及依失敗脈絡檢索之修復導向經驗。系統接著將 T 、既有計畫 P 、失敗脈絡 F 與 $\mathcal{E}_{\text{rp}}$ 交由重新規劃器產生後續計畫。重新產生的計畫會更新 P ，再交回任務執行流程，使系統繼續執行尚未完成的任務。

3. **結果回傳：輸出執行結果。**當目前計畫 P 中已無尚未執行的步驟，或任務流程到達終止條件時，系統回傳最終執行結果 R 。後續實驗章節將依據 AppWorld 評估結果判斷 R 是否符合任務需求。

整體而言，CBR-APIKGAgent 採取「先局部修復、必要時重新規劃」的錯誤處理流程。此流程的設計目的並非取代重新規劃，而是讓系統在錯誤可能由小範圍計畫調整處理時，先嘗試保留原本仍有效的計畫與資料流程；當局部修復無法形成可執行修改，或修復嘗試達到限制時，再回退至重新規劃。由於本研究不假設任務環境可回復至失敗前狀態，重新規劃與局部修復皆是在目前已觀察到的執行狀態下進行。透過原子化經驗檢索與動態局部修復的結合，本研究進一步探討案例式策略參考是否有助於改善多步驟 API 任務中的規劃品質與錯誤修復表現。

3.2 原子化經驗重用機制

本節對應演算法 1 中的經驗檢索步驟，說明原子化經驗如何由訓練資料中的成功任務軌跡建立，並於任務流程中被表示、檢索與使用。此機制並非獨立執行的子系統，

而是透過語意檢索與提示注入，在初始規劃、局部修復與重新規劃階段提供策略參考。

原子化經驗重用機制的目的，是將成功任務軌跡中隱含的規劃與資料流策略，轉換為可跨任務重用的經驗知識。相較於直接保存完整任務軌跡，本研究將經驗拆解為較小的策略單元，使其能提醒代理人注意相似任務中的資料依賴、操作順序與常見陷阱，而非要求代理人複製過去任務的完整步驟。以下先說明經驗來源與原子化經驗定義，再說明其在不同決策階段的檢索與使用方式。

3.2.1 經驗來源與使用範圍

由成功任務證據建立並重用原子化經驗的概念性流程如圖 3.2 所示。

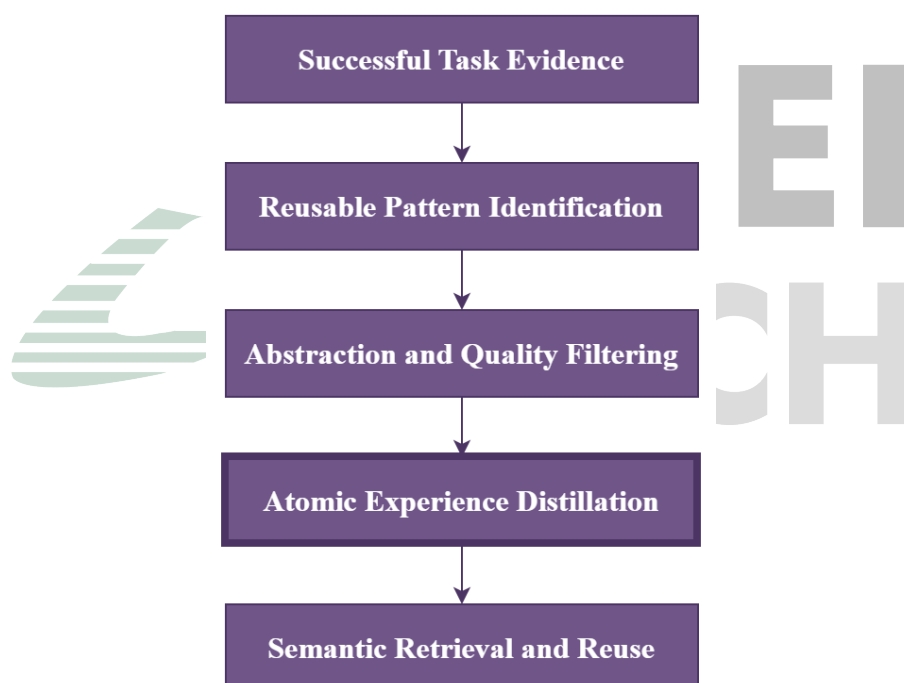


圖 3.2: 原子化經驗萃取與重用流程

圖 3.2 呈現的是由成功任務證據轉換為可重用經驗的概念性處理流程，而非五個彼此獨立的軟體模組。系統首先彙整訓練任務的任務目標、公開資料、評估條件、標準解答與 API 呼叫證據，從中辨識具有跨任務價值的目標集合、資料依賴、操作限制與規劃原則。接著，系統移除固定參數及其他任務限定資訊，並進行品質篩選；若候選內容僅能描述特定任務配方，或缺乏明確的重用價值，則不產生經驗。通過篩選的模式會被萃取為原子化經驗，而非保留完整的 API 呼叫序列，後續再透過語意索引提供

初始規劃、重新規劃與局部修復階段參考。相關實作細節於第四章說明。

本研究使用的正式經驗庫由 AppWorld training split 中具有標準解答與成功執行證據的任務資料離線建立。測試期間不新增經驗至正式經驗庫，也不更新正式檢索引。此設計使經驗來源與實驗評估資料保持切割，避免測試期間產生的執行紀錄影響後續任務評估。

雖然系統在局部修復過程中可能產生新的修復軌跡，但此類軌跡不會被納入正式經驗庫。主要原因在於，單一次局部修復不一定代表整體任務成功，且其正確性需由後續資料流與最終任務結果共同判斷。因此，本研究不將測試期間產生的修復軌跡轉換為可檢索引，以避免未驗證內容影響後續規劃與修復決策。

3.2.2 原子化經驗定義

本研究將原子化經驗（Atomic Experience）定義為一個可跨任務重用的最小策略單元，用以描述特定任務情境下應保留的規劃原則、資料流模式、必要檢查與常見陷阱。換言之，本研究並非直接重用完整案例，而是將案例中的可重用策略整理為原子化經驗，作為後續檢索引與提示注入的基本單位。

一筆原子化經驗通常對應到一類可重複出現的規劃需求或錯誤風險，其重點不在於記錄某次任務使用了哪些 API，而在於保留該任務成功時必須滿足的資料流與檢查原則。例如，若某類任務需要更新符合條件的多筆資料，原子化經驗關注的不是特定 API 呼叫與固定參數，而是「先建立符合條件的目標集合，再確認各目標項目的狀態後執行更新」這類可重用的資料流原則。

為明確界定原子化經驗在本研究中的角色，以下從其保存內容與使用方式說明其定位：

- **案例粒度**：原子化經驗不重播完整任務軌跡，而是抽取其中可跨任務重用的策略片段。
- **操作層級**：原子化經驗不保存固定 API 呼叫序列或參數值，而是描述資料流、檢查條件與決策原則。
- **使用方式**：部分代理人系統會將可重用能力封裝為技能、工具或子程序；本研究

的原子化經驗則著重於以自然語言形式保存細粒度策略知識，並透過檢索提供給模型作為規劃與修復參考，而非作為可直接調用的執行模組。

為了使上述經驗能被檢索、排序並注入不同決策階段，本研究進一步將每筆原子化經驗表示為結構化知識。表 3.2 以概念層級整理原子化經驗所保留的主要資訊面向，用以說明成功任務軌跡如何被抽象化為可檢索經驗；實際儲存格式、欄位名稱與索引使用方式則於第四章系統實作中說明。

表 3.2: 原子化經驗概念組成

資訊面向	概念用途
來源脈絡	保留經驗所來自的成功任務背景，使系統能理解該經驗原先適用的任務目標與資料情境。
適用情境	描述此經驗適合被參考的任務或錯誤情境，例如目標集合建構、候選驗證、批次處理或輸出格式限制。
核心原則	保留成功任務中可跨任務重用的規劃原則或資料流不變條件。
必要中間概念	說明規劃或修復過程中應維持的中間資料、目標集合、驗證條件或操作順序。
套用方式	描述將經驗轉移至新任務時可參考的抽象步驟，作為規劃或修復時的策略提示，而非固定 API 呼叫序列。
常見陷阱	指出此類任務中容易發生的錯誤，例如過早執行寫入、忽略篩選條件、不完整取得資料或使用錯誤目標集合。

上述資訊面向屬於方法設計層級的概念組成，而非系統中的實際資料欄位。透過這樣的結構化表示，原子化經驗能同時保留來源背景、適用條件、可重用策略與錯誤避免資訊。

範例：原子化經驗概念示例

以一筆用於經驗萃取的訓練任務軌跡為例，該任務要求代理人在 Venmo 社交動態中，找出「昨天」且「交易參與者包含任一位兄弟姐妹」的交易，並對這些交易按讚。此任務軌跡可萃取出多筆原子化經驗；其中一筆經驗屬於寫入目標保護類型，重點在於避免代理人在尚未確認最終目標集合前執行寫入動作，其內容可整理如下：

- **適用情境：**當某個動作需要套用於大型集合中的特定子集合，且該子集合由多個彼此獨立的篩選條件共同定義時。
- **核心原則：**寫入動作只能套用於符合所有指定篩選條件的項目，以避免不必要或錯誤的資料修改。

- **必要中間概念：**初始候選項目集合、篩選條件集合，以及套用所有條件後形成的最終操作目標集合。
- **套用方式：**先從資料來源取得完整的潛在目標集合，接著根據任務需求定義所有必要篩選條件，逐一檢查每個候選項目是否符合條件，最後只對符合所有條件的項目執行寫入動作。
- **常見陷阱：**在所有篩選條件檢查完成前就執行寫入、錯誤定義或套用篩選條件、漏掉關鍵條件，或誤以為單一條件即可決定最終目標集合。

上述資訊面向使原子化經驗同時具備可追溯性、可檢索性與可注入性。整體而言，原子化經驗以較抽象的資料來源、候選集合、驗證條件、目標集合與輸出契約等概念描述可重用策略，避免過度貼近單一訓練任務。

3.2.3 經驗檢索視角與使用情境

原子化經驗雖來自同一份經驗庫，但在不同決策階段所需強調的語意重點並不相同。為使經驗能同時支援規劃與修復，本研究將經驗檢索區分為規劃導向與修復導向兩種視角：

- **規劃導向檢索：**主要強調適用情境、核心原則、套用方式與任務目標，使經驗能支援任務結構、資料依賴或操作順序相關的規劃判斷。
- **修復導向檢索：**主要強調適用情境、核心原則、套用方式與常見陷阱，使經驗能支援失敗訊息、錯誤脈絡或修復方向相關的判斷。

此設計並不代表系統維護兩套不同來源的經驗，而是使同一批原子化經驗能依決策目的凸顯不同資訊。實際索引建立、查詢格式與提示注入方式於第四章說明。

依照使用階段不同，原子化經驗主要支援以下三種決策情境：

1. 初始規劃階段

- **使用視角：**此階段主要使用規劃導向經驗。

- **設計目的：**在初始計畫生成前，提醒代理人注意相似任務中的資料流條件、目標範圍與常見陷阱，使計畫能較早納入必要中間概念與操作順序。

2. 局部修復階段

- **使用視角：**此階段主要使用修復導向經驗。
- **設計目的：**在步驟失敗後，提供與目前失敗情境相近的資料流原則與常見陷阱，輔助判斷應補入前置資料、替換失敗操作，或移除不必要的步驟。

3. 重新規劃階段

- **使用視角：**此階段同時使用規劃導向與修復導向經驗。
- **設計目的：**重新規劃需同時回應原始任務需求與已發生的失敗，因此需兼顧任務結構層面的規劃經驗，以及錯誤情境層面的修復經驗。

整體而言，原子化經驗重用機制提供的是可依不同決策情境檢索的案例式知識，而非固定規則或保證正確的修復模板。

3.3 動態局部修復機制

本節對應演算法 1 中步驟執行失敗後的錯誤處理分支，說明 CBR-APIKGAgent 如何透過局部計畫修改嘗試恢復任務執行。由於整體任務流程已於前述演算法說明，本節聚焦於局部修復的設計目標、局部修復與重新規劃之選擇原則，以及插入、替換與刪除三種計畫修改策略。

3.3.1 局部修復設計目標與適用邊界

動態局部修復（Dynamic Local Repair）的設計目標，是在任務執行過程中發生局部失敗時，根據當前可取得的失敗步驟、錯誤訊息、相依資料與執行結果形成局部調整方案，而非立即重新產生整體計畫。此設計背後的假設是：多步驟 API 任務中的失敗不一定需要整體重新規劃處理，有時可透過補足前置資料、改用較合適的操作方式，

或移除不必要的重複步驟進行局部修正。在此情況下，局部修復可保留已成功執行的步驟與仍有效的中間資料，並降低重新規劃對原本正確資料流程造成干擾的可能性。

由於任務執行可能已改變外部應用程式狀態，此修復流程不假設可回復至錯誤前狀態，而是依據目前可取得的失敗資訊形成局部調整方案。然而，局部修復並不適用於所有失敗情境。若失敗被暫定分類為不可行計畫，或同一原始失敗點已達修復嘗試限制，系統應回退至重新規劃。換言之，局部修復在本研究中被定位為重新規劃前的局部調整機制，而非取代重新規劃的完整錯誤處理方法。

3.3.2 局部修復與重新規劃之選擇原則

局部修復與重新規劃的選擇，決定了系統在失敗發生後應保留原計畫並局部調整，或重新產生後續計畫。此選擇並非建立在精確的錯誤根因判斷上，而是根據步驟失敗後可取得的有限資訊進行保守決策。這些資訊包含失敗步驟的位置與類型、暫定錯誤分類、錯誤訊息、可行回饋，以及同一原始失敗點的修復嘗試狀態。基於上述資訊，本研究採用下列選擇原則：

- **優先保留局部修復空間：**當失敗看似侷限於單一步驟附近，例如缺少前置資料、操作方式不合適，或局部步驟結果不足以支撐後續計畫時，系統優先嘗試局部修復，以保留已成功執行的步驟與既有資料流程。
- **避免無限制反覆修復：**若修復步驟本身再次失敗，該失敗會被視為原始失敗點的延伸，而不是新的獨立失敗點。當同一原始失敗點達到修復嘗試限制時，系統停止產生新的局部修復方案。
- **保留重新規劃作為回退機制：**若錯誤被暫定為不適合局部處理，或局部修復無法形成有效計畫修改，系統回退至重新規劃。此設計使局部修復可作為重新規劃前的嘗試，而不取代重新規劃本身。

可行回饋不直接決定上述選擇，而是在進入局部修復後作為修復行動的依據。相關限制的具體紀錄方式與流程銜接於第四章說明。

3.3.3 局部計畫修復策略

在系統判斷應嘗試局部修復後，局部計畫修復策略的目標，是根據目前失敗步驟與相關錯誤資訊產生一個可套用於原計畫片段的修改方案。此修改方案以最小化計畫變動為原則，僅調整失敗步驟附近的內容，並盡可能保留已成功執行的步驟與可用中間資料。

需要注意的是，局部修復策略的輸出是修復後的計畫片段，而非立即完成 API 執行；修復步驟被加入、替換或刪除後，系統會回到對應位置接續執行更新後的計畫。

本研究將局部計畫修改方式分為插入步驟（Insert）、替換步驟（Replace）與刪除步驟（Delete）三類。為了說明三種策略在計畫序列上的差異，令目前計畫可表示為：

$$P = \langle S_1, S_2, \dots, S_k^{fail}, \dots, S_n \rangle$$

其中， S_k^{fail} 表示目前執行失敗的步驟。對應演算法 1，三種策略皆可視為 $\text{GenerateRepairPlan}(P, s, m)$ 依據目前計畫 P 、失敗步驟 s 與局部修改方式 m ，產生修復後計畫 P_{repair} 的不同形式。以下分別說明三種策略的適用情境、計畫調整方式與使用原則。

此處索引用於標示步驟在原計畫中的相對位置；修復後計畫的實際步驟順序與編號可於計畫更新時重新整理。因此， $S_{k+1}, \dots, S_n$ 表示原計畫中位於失敗步驟之後的後續步驟，而非修復後計畫的實際最終編號。

以下各策略皆以前述 P 作為原始計畫，並列出修復後計畫 P_{repair} 的形式。

1. 插入步驟（Insert）

- **適用情境：**當原步驟本身仍可能正確，但執行前缺少必要輸入或前置狀態時，系統會考慮在原失敗步驟前插入新的修復步驟。
- **計畫調整：**新增修復步驟會被安排在原失敗步驟之前，原失敗步驟仍保留於計畫中，並於修復步驟完成後再次執行。此策略可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, R^{ins}, S_k, S_{k+1}, \dots, S_n \rangle$$

其中， R^{ins} 表示插入策略產生的修復步驟，其上標用於標示修復策略類型，並不代表修復後計畫中的實際步驟編號。此步驟用於補足原步驟所需的前置資料或狀態； S_k 則代表原失敗步驟在修復後重新進入執行流程。

- **使用原則：**插入策略的重點在於保留原失敗步驟的角色。新增步驟應用於補足資料或建立前置條件；若候選 API 與原步驟具有相同操作目的，則不應以插入方式加入，以避免同一操作被重複執行，進而造成資料狀態被重複修改或與任務目標不一致。

2. 替換步驟 (Replace)

- **適用情境：**當原失敗步驟所代表的局部目標仍需完成，但其 API 選擇、操作方式、目標範圍或步驟類型不適當時，系統會考慮以新的修復步驟替換原步驟。
- **計畫調整：**新的修復步驟會取代原失敗步驟，原失敗步驟不再於後續流程中執行。此策略保留原步驟在計畫中的位置，但改以新的執行方式完成該位置原本應達成的局部目標，可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, R^{rep}, S_{k+1}, \dots, S_n \rangle$$

其中， R^{rep} 表示替換策略產生的修復步驟，其上標用於標示修復策略類型，並不代表修復後計畫中的實際步驟編號。此步驟接替原失敗步驟在計畫中的位置，並承擔其原本應完成的局部目標。

- **使用原則：**替換策略的目的並非改變原計畫的任務語意，而是以另一種方式完成原失敗步驟的局部目標。由於替換可能影響後續資料流，系統僅在失敗證據支持原步驟不宜再次執行時採用此策略，並傾向保留原步驟的相依資料；若候選 API 僅能提供前置或輔助資料，則不應直接取代原步驟。

3. 刪除步驟 (Delete)

- **適用情境：**當失敗資訊顯示該步驟可能為冗餘、重複，或其局部目標已由前序步驟完成而不需再次執行時，系統會考慮將其從局部計畫中移除。此策略

適用於保留該步驟反而可能造成重複操作、重複資料修改或干擾後續流程的情境。

- **計畫調整：**系統會移除原失敗步驟，並使刪除後的計畫能銜接前後步驟繼續執行。此策略可表示為：

$$P_{\text{repair}} = \langle S_1, \dots, S_{k-1}, S_{k+1}, \dots, S_n \rangle$$

其中，原失敗步驟 S_k^{fail} 不再出現在修復後計畫中。

- **使用原則：**刪除策略主要處理計畫結構上的銜接，並不額外產生被刪除步驟原本可能提供的資料。因此，若後續執行過程顯示刪除造成必要資料不足，該失敗仍需透過後續局部修復或重新規劃處理。

整體而言，上述公式描述的是計畫序列層級的變化；實際執行時，系統仍需維持前後步驟與資料依賴關係的一致性。對應演算法 1，當 P_{repair} 產生後，系統會以其更新目前計畫 P ，並透過 $\text{SetNextStepToRepairPoint}(P)$ 將後續執行位置調整至修復後計畫中需要接續執行的位置。若後續執行仍出現資料不足或目標不符，系統會再依當前失敗狀態進入局部修復或重新規劃流程。

第四章 CBR-APIKGAgent 系統實作

本章說明 CBR-APIKGAgent 的系統實作方式，重點在於原子化經驗重用與動態局部修復如何落實於實際任務流程中。相較於第三章著重方法設計與機制原理，本章進一步說明各模組在系統執行時的資料傳遞、提示組合與流程銜接方式。

以下依系統運作順序，分別說明原子化經驗庫建構、初始規劃、步驟執行、失敗處理與重新規劃等實作流程。

4.1 原子化經驗庫建構實作

原子化經驗庫於任務執行前建立，作為後續初始規劃、局部修復與重新規劃階段的經驗來源。本節聚焦於經驗庫如何由訓練資料成功任務軌跡建構；各階段之經驗檢索與使用方式，將於後續對應流程中分別說明。

4.1.1 訓練資料成功任務軌跡萃取

原子化經驗的萃取原則與概念性流程已於第三章圖 3.2 說明。本節聚焦其實作方式：經驗轉換模組於任務執行前離線讀取訓練任務資料、整理成功任務證據，並將其組合為萃取提示的輸入，再由大型語言模型輸出結構化原子化經驗。每個來源任務可產生零至兩筆經驗，原則上優先產生一筆；僅在證據包含兩項明確且彼此獨立的可重用模式時，才產生兩筆。

實作上，每個訓練任務的經驗萃取證據由 `specs.json`、`public_data.json`、`solution.py`、`evaluation.py`，以及壓縮後的 `api_calls.json` 組成；其中 API 呼叫紀錄保留主要呼叫方法、網址與資料欄位名稱。本研究不使用 `private_data.json`、`test_data.json` 或任務執行日誌進行經驗萃取。最終由 90 個訓練任務產生 160 筆原子化經驗，作為初始規劃、局部修復與重新規劃階段的檢索來源；完整經驗萃取 Prompt、輸出格式與實際範例如附錄 A.1 所示。

4.1.2 原子化經驗資料格式

原子化經驗在系統中以 JSON Lines (JSONL) 格式保存，每一行對應一筆經驗物件。依後續索引建構與提示注入的需求，每筆經驗可分為下列三類資訊：

- **來源與類型資訊**：說明經驗的類型與來源任務背景，主要包含下列欄位：
 - 經驗類型 (experience_type)：標示經驗所屬類別。
 - 來源任務編號 (source_task_id)：用於來源追溯，不直接作為語意檢索或提示注入的主要內容。
 - 來源任務目標 (source_task_goal)：保留來源任務語意，使經驗可追溯至原任務脈絡。
- **策略內容資訊**：描述經驗可被重用的條件、原則與注意事項，主要包含下列欄位：
 - 適用情境 (when_to_use)：描述此經驗適合被參考的任務或錯誤情境。
 - 核心原則 (core_principle)：保存可跨任務重用的資料流原則或規劃原則。
 - 套用方式 (how_to_apply)：說明將此經驗轉移至目前任務時可參考的操作方向。
 - 常見陷阱 (pitfalls)：記錄此類情境中容易導致錯誤的資料依賴、篩選或執行順序問題。
- **中間概念資訊**：說明任務成功過程中需要建立或維持的關鍵中間資料：
 - 必要中間概念 (required_intermediate_concepts)：記錄目標集合、候選項目、狀態檢查結果或輸出約束等中間資料。系統在提示中主要呈現概念名稱與角色說明，使模型注意必要中間資料。

上述欄位使每筆原子化經驗同時保留來源脈絡、策略內容與必要中間概念。系統後續會依不同使用情境選取並重組欄位內容，而非直接將原始 JSON 物件作為決策輸入。

4.1.3 經驗向量化與雙索引建構

經驗檢索由語意檢索模組實作。系統建置經驗索引時，會讀取經驗檔案，並為同一批經驗建立規劃導向與修復導向兩組索引向量。兩組索引向量使用相同經驗來源，但會依不同使用情境選取欄位並組成不同索引文字：

- **規劃導向索引**：用於任務規劃相關情境，主要包含下列欄位：

- when_to_use
- core_principle
- how_to_apply
- source_task_goal

- **修復導向索引**：用於失敗處理與修復相關情境，主要包含下列欄位：

- when_to_use
- core_principle
- how_to_apply
- pitfalls

採用雙索引的目的包含以下兩點：

- **區分語意比對焦點**：規劃與修復階段所需比對的語意焦點不同。規劃階段較重視來源任務目標、適用情境與規劃原則；修復階段則較重視失敗情境、可套用原則與常見陷阱。
- **減少重複向量化**：兩組索引向量會在任務執行前預先建立，使任務執行期間可直接依目前階段選擇對應索引進行查詢，而不需在每一次規劃或修復決策時重新組合經驗內容並產生向量表示。

檢索時，系統會將查詢文字轉換為向量表示，並以餘弦相似度衡量查詢向量與索引中經驗向量之間的語意接近程度。

4.2 初始規劃實作

任務開始後，系統先根據使用者任務搜尋候選 API，並結合規劃導向經驗產生初始任務計畫。本節說明初始規劃階段中 API 搜尋、經驗檢索與提示注入的實作方式。

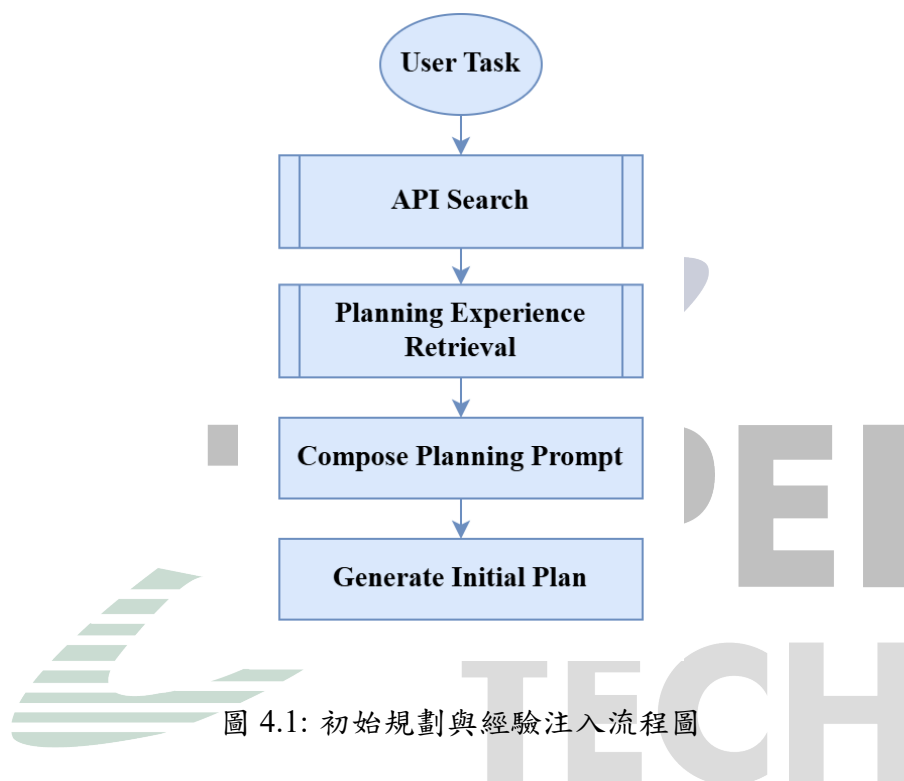


圖 4.1: 初始規劃與經驗注入流程圖

圖 4.1 呈現初始規劃階段中，API 搜尋與規劃導向經驗檢索如何分別提供工具資訊與策略參考，並共同形成任務規劃提示。

此階段可整理為下列三個主要步驟：

- **API 搜尋與候選整理：**根據使用者任務查詢 API 知識圖譜，取得候選 API 與其使用資訊。
- **規劃導向經驗檢索：**根據使用者任務查詢規劃導向索引，取得與任務結構相關的原子化經驗。
- **提示組合與計畫生成：**將使用者任務、候選 API 與規劃導向經驗組合為初始規劃提示，並由大型語言模型產生初始任務計畫。

4.2.1 規劃導向經驗檢索

初始規劃階段以使用者任務作為查詢，透過規劃導向索引取得相關原子化經驗，並將檢索結果格式化為精簡策略提醒。

此處的查詢主要由原始使用者任務與任務目標語意組成；系統會將其與規劃導向索引中的來源任務目標、適用情境、核心原則與套用方式進行語意比對，以取得與當前任務結構相近的經驗。

檢索結果主要聚焦於相似任務中的資料依賴、目標集合建構、候選驗證或輸出格式限制等問題，使規劃階段能取得與當前任務結構相關的經驗內容。此階段的輸出會於後續提示組合時提供給任務規劃器使用。

4.2.2 初始計畫生成與提示注入

初始規劃提示包含使用者任務、候選 API 以及規劃導向經驗。候選 API 由 APIKGAgent 的 API 知識圖譜搜尋流程取得，並以 API 名稱、功能描述、參數需求與回傳結構等形式提供可用工具範圍；規劃導向經驗則提供資料流與規劃策略參考。系統會將兩者一同納入提示，使大型語言模型規劃器能在可用 API 範圍內，參考相似任務中的策略經驗產生初始計畫。

在提示內容上，初始規劃階段主要提供下列資訊：

- **任務需求：**使用者輸入的自然語言任務，作為計畫產生的主要目標。
- **候選 API 資訊：**包含 API 名稱、功能描述、參數需求與回傳結構，用以限制規劃器可使用的工具範圍。
- **規劃導向經驗：**包含與任務結構相關的適用情境、核心原則、套用方式、必要中間概念與常見陷阱，提供規劃時的策略參考。
- **計畫輸出要求：**要求規劃器產生可逐步執行的計畫，並明確區分 API 呼叫、資料處理與任務完成等步驟類型。

本研究在初始規劃階段提供至多指定數量的規劃導向經驗；其數量上限與相似度門檻屬於實驗控制條件，於第五章實驗設置中說明。經驗注入片段如附錄 A.4 所示。

提示會要求模型將經驗視為精簡策略參考，而非任務特定解法或固定 API 呼叫序列。尤其當經驗包含集合、物件或中間概念等描述時，提示會要求模型將其理解為資料流語意，而不是直接套用為特定資料型態或固定輸出格式。此設計可降低模型將經驗誤用為具體程式模板的風險。

初始計畫產生後，系統會將其交由步驟執行流程逐步執行，並於後續階段依執行結果更新任務狀態或進入失敗處理流程。

4.3 步驟執行與任務狀態管理

產生初始計畫後，系統依序執行計畫中的步驟。此階段的重點在於保存已完成步驟的執行結果、維持步驟間的資料依賴，並在失敗發生時提供後續錯誤處理所需的上下文。

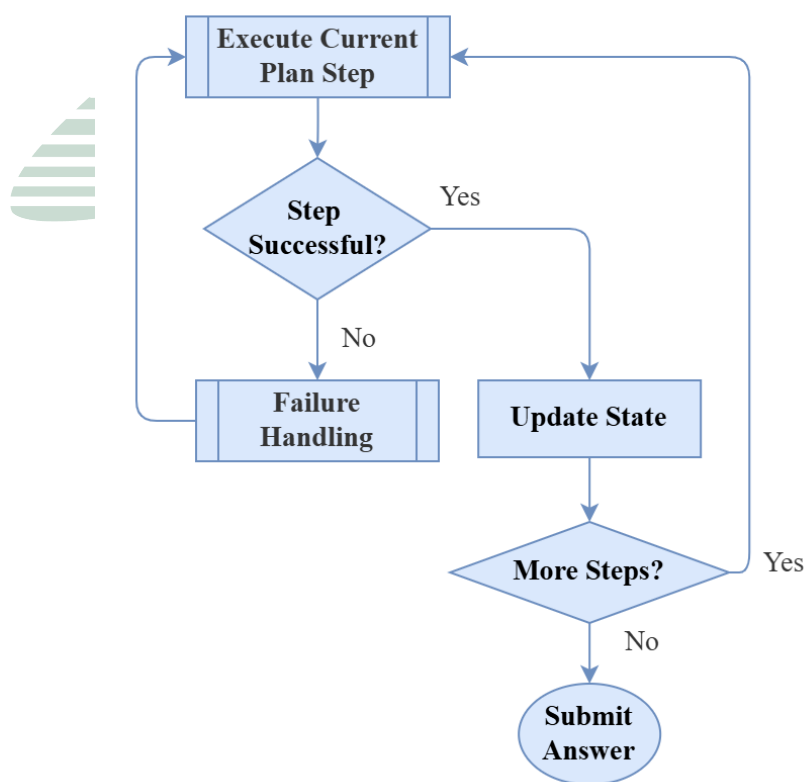


圖 4.2: 步驟執行與狀態更新流程圖

圖 4.2 呈現任務計畫進入逐步執行後，系統如何依步驟類型執行動作、保存成功結果，並在失敗時將錯誤資訊交由後續失敗處理流程使用。

此階段可整理為下列三類主要狀態管理工作：

- **取得待執行步驟與相依資料：**根據目前任務計畫與執行位置，取得待執行步驟，並讀取該步驟所需的前序結果。
- **保存成功步驟輸出：**當 API 呼叫或資料處理成功後，將結果寫回任務狀態，使後續步驟與修復流程可追蹤資料來源。
- **保留失敗脈絡：**當步驟失敗時，保存失敗步驟、錯誤訊息、已完成步驟結果與相關執行紀錄，作為後續失敗處理的輸入。

4.3.1 執行結果與相依資料保存

系統主流程由 LangGraph 狀態圖控制。步驟執行階段的輸入為目前任務計畫與待執行步驟；系統會根據目前步驟類型執行 API 呼叫或資料處理動作，並將執行結果寫回任務狀態。具體而言，每次步驟執行後會留下下列原始執行資訊：

- **成功執行結果：**包含 API 回傳資料或資料處理結果，作為後續步驟的相依資料。
- **失敗步驟資訊：**包含失敗步驟在計畫中的位置、步驟描述，以及該步驟原本欲完成的局部目標。
- **原始錯誤訊息：**包含 API 呼叫失敗、資料處理例外、回傳內容不符合預期，或步驟無法完成時產生的錯誤文字。
- **已完成步驟脈絡：**包含先前成功步驟的輸出與資料來源，使後續失敗處理流程能取得錯誤發生前的資料狀態。

LangGraph 的狀態傳遞機制使各節點能共享任務目標、目前計畫、候選 API、執行結果與錯誤資訊。此設計使 CBR-APIKGAgent 能在同一任務狀態下銜接規劃、執行與後續失敗處理；其中，步驟執行階段僅負責保存原始執行結果與失敗脈絡，後續的錯誤分類、可行回饋與局部修復則由失敗處理流程負責。

4.4 失敗處理與局部修復實作

當步驟執行失敗時，系統會先整理錯誤資訊與相依資料，產生暫定錯誤分類與可行回饋；除不可行計畫或修復限制已達上限等明確回退條件外，系統會先嘗試局部修復，包含搜尋修復 API、檢索修復導向經驗，並產生插入、替換或刪除等局部計畫修改。

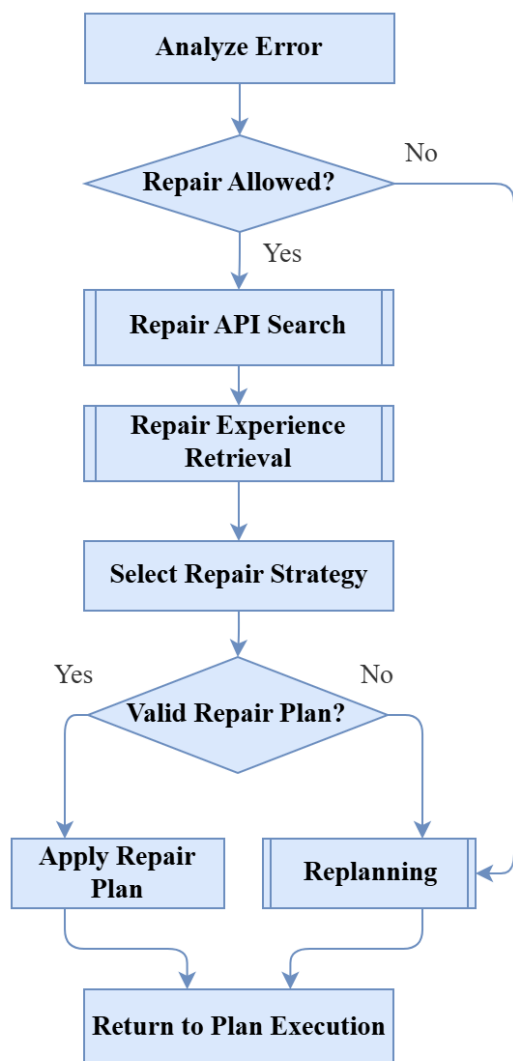


圖 4.3: 局部修復與重新規劃回退流程圖

圖 4.3 呈現步驟失敗後的錯誤處理路由，特別是錯誤分類、修復限制檢查、修復 API 搜尋、修復導向經驗檢索、策略選擇與重新規劃回退之間的關係。

此階段可整理為下列五個主要步驟：

- **錯誤分類與可行回饋產生**：整理失敗步驟、錯誤訊息與相依資料，並由大型語言模型產生暫定錯誤分類與可操作的修復建議。
- **局部修復觸發與限制檢查**：根據暫定錯誤分類與修復嘗試狀態，檢查是否符合直接回退至重新規劃的條件；若未觸發回退條件，則進入局部修復流程。
- **修復 API 搜尋與候選整理**：根據失敗資訊與可行回饋搜尋可能用於修復的候選 API。
- **修復導向經驗注入**：檢索與目前錯誤情境相關的修復導向經驗，提供策略選擇時參考。
- **局部計畫修改**：由大型語言模型根據候選 API、修復經驗與失敗證據選擇插入、替換或刪除策略，產生修復後計畫或回退至重新規劃。

除了由步驟執行階段傳入的原始失敗資訊外，失敗處理與局部修復流程會進一步產生並保存錯誤分析、修復嘗試與局部計畫修改相關狀態。表 4.1 以概念層級整理本階段主要使用的狀態資訊。

表 4.1: 錯誤處理與局部修復相關狀態資訊

資訊類型	用途	主要使用階段
暫定錯誤分類	保存錯誤分類流程產生的暫定類型。	失敗處理輸入
可行回饋	保存由錯誤訊息整理而成的可操作修復建議。	失敗處理輸入
結構化失敗資訊	保存失敗 API 與執行結果分析形成的結構化失敗資訊。	失敗處理輸入
修復嘗試紀錄	記錄同一原始失敗點已進行的局部修復嘗試。	修復狀態追蹤
修復步驟關聯	記錄修復步驟與原始失敗步驟之間的關係。	修復狀態追蹤
局部修復紀錄	保存同一任務中局部修復嘗試的摘要，例如修復對象、修復方式、候選 API 與相關失敗證據，用於追蹤修復狀態與避免重複修復。	修復狀態追蹤

4.4.1 錯誤分類與可行回饋產生

當任務執行器回報步驟失敗時，系統會先整理失敗步驟、失敗 API、原始錯誤訊息、已完成步驟結果與相依資料脈絡，形成錯誤分析提示。此提示會交由大型語言模

型產生兩項輸出：暫定錯誤分類與可行回饋。暫定錯誤分類由預先定義的錯誤分類集合中選出，用於描述目前失敗較接近的錯誤情境；可行回饋則將錯誤訊息整理為後續可操作的修復方向。完整提示詞與錯誤分類集合如附錄 A.2 所示。

若執行器提供結構化錯誤脈絡，錯誤分析提示亦會納入目前記憶狀態已有的欄位 (`current_memory_keys`)、候選 API (`suggested_apis`)、失敗呼叫實際使用的參數 (`params_used`)、相關 API 規格片段 (`api_doc_snippet`)，以及 API 或執行器回傳的錯誤細節 (`details`)，使模型能對照實際參數與 API 要求形成較具體的可行回饋。

此分類結果不作為精確根因判斷，而是用於後續錯誤處理路由與修復輔助。可行回饋則會被用於修復 API 搜尋與策略選擇，使原始錯誤訊息能轉換為較具操作性的修復線索。

4.4.2 局部修復觸發與限制檢查

錯誤處理路由由條件路由流程控制。此階段並不依賴模型精確判斷錯誤根因，而是根據暫定錯誤分類與修復狀態，決定是否先嘗試局部修復或回退至重新規劃。主要限制如下：

- **不可行計畫直接重新規劃：**若原始失敗步驟被暫定分類為不可行計畫，系統會直接回退至重新規劃。此類情境表示目前計畫中的該步驟缺乏可執行的工具或操作路徑，例如計畫要求執行某項操作，但目前 API 搜尋結果中沒有足以支持該操作的相關 API。
- **同一失敗點修復次數限制：**系統會追蹤同一原始失敗步驟已嘗試的局部修復次數。若修復步驟本身再次失敗，後續修復嘗試仍會透過修復步驟與原始失敗步驟的對應關係，累計至同一原始失敗點；當累計次數達到上限時，不再繼續局部修復，而是回退至重新規劃。具體次數上限作為實驗控制設定，於第五章說明。
- **修復鏈深度限制：**除了累計修復次數外，系統也限制同一原始失敗點可形成的修復鏈深度。此限制主要用於避免同一錯誤位置因連續插入修復步驟而使計畫不斷擴張；具體深度上限作為實驗控制設定，於第五章說明。

在上述限制均未觸發時，系統會先進入局部修復流程。此設計使局部修復能作為重新規劃前的有限嘗試，同時避免對同一錯誤位置進行無限制的計畫修改。

4.4.3 修復 API 搜尋與候選整理

系統會使用失敗訊息與可行回饋組成修復搜尋目標，並呼叫 API 搜尋流程取得候選修復 API。候選 API 會被整理為 API 名稱、描述與回傳結構等資訊，提供給後續修復策略選擇。

此階段的目的不是直接修改計畫，而是取得可用工具。是否使用候選 API，以及候選 API 應插入、替換或刪除原步驟，仍由後續策略選擇流程決定。

4.4.4 修復導向經驗注入

取得候選 API 後，局部修復策略選擇流程會以失敗訊息查詢修復導向索引，取得與目前錯誤情境相關的經驗。相較於初始規劃階段以任務目標尋找相似任務結構，局部修復階段的查詢重點在於目前失敗所呈現的錯誤脈絡，例如錯誤訊息中反映的資料缺漏、參數不一致、API 使用不當或操作順序問題。

在提示內容上，局部修復策略選擇階段主要提供下列資訊：

- **失敗步驟資訊：**包含原計畫中失敗步驟的描述、步驟位置與其原本欲完成的局部目標。
- **失敗證據：**包含原始錯誤訊息、失敗 API 資訊、結構化失敗資訊，以及已完成步驟所留下的相依資料脈絡。
- **可行回饋：**由前一階段錯誤分析產生，用於將原始錯誤訊息轉換為較具操作性的修復方向。
- **候選修復 API：**由修復 API 搜尋取得，提供模型可選擇的工具範圍。
- **修復導向經驗：**包含與目前失敗情境相關的適用情境、核心原則、套用方式與常見陷阱，作為策略選擇時的參考。

檢索到的修復經驗不直接決定修復策略，而是與上述資訊一同提供給大型語言模型，使模型在選擇插入、替換或刪除策略時，能參考相似錯誤情境中的資料流原則與常見陷阱。本研究在局部修復階段提供至多指定數量的修復導向經驗；其數量上限與相似度門檻屬於實驗控制條件，於第五章實驗設置中說明。

4.4.5 插入、替換與刪除策略實作

前述失敗步驟資訊、失敗證據、可行回饋、候選 API 與修復導向經驗會共同形成局部修復策略選擇提示。大型語言模型會根據此提示產生結構化輸出，包含欲使用的修復 API、是否沿用原失敗步驟的前序相依資料、計畫修改方式與修復意圖。修復意圖分為缺少前置資料 (MISSING_PREREQUISITE)、補足未完整執行 (COMPLETE_PARTIAL_EXECUTION)、修正錯誤操作 (CORRECT_WRONG_OPERATION) 與移除冗餘步驟 (REMOVE_REDUNDANT_STEP)；四者分別對應插入、替換、替換與刪除策略。完整提示詞與輸出格式如附錄 A.3 所示。系統不直接執行模型輸出的文字說明，而是將其轉換為下列計畫更新流程：

- **解析策略輸出：**系統先檢查模型輸出的計畫修改方式是否屬於插入、替換或刪除，並確認模型是否選出可用的修復 API；若輸出無法對應到可套用的策略或未選出修復 API，則不修改目前計畫。
- **建立或移除修復步驟：**若策略為插入或替換，系統會依所選 API 建立新的修復步驟，並依模型輸出的相依設定決定是否沿用原失敗步驟的前序資料；若策略為刪除，則移除原失敗步驟且不新增 API 步驟。
- **更新計畫結構：**系統依策略調整目前計畫中的步驟順序、步驟編號與後續執行位置。插入策略會使新增修復步驟先被執行，替換策略會以新步驟接替原位置，刪除策略則使後續步驟補位銜接。
- **更新修復追蹤狀態：**系統會記錄修復步驟與原始失敗步驟之間的對應關係，並更新同一原始失敗點的修復嘗試紀錄，使後續若修復步驟再次失敗，仍可回掛至原始失敗點進行限制檢查。

若模型輸出可轉換為上述計畫更新，系統會更新目前計畫並回到步驟執行流程；若輸出缺少必要修復 API、無法對應至可套用策略，或無法形成有效的計畫修改，則不套用局部修復結果，並回退至重新規劃。

4.5 重新規劃實作

重新規劃是失敗處理流程中的回退路徑，可能在錯誤分類後、修復限制檢查後、修復 API 搜尋後，或局部修復無法成功更新目前計畫時被觸發。重新規劃的目的，是在保留原始任務目標、既有計畫、已完成步驟與失敗資訊的基礎上，重新產生後續可執行計畫。

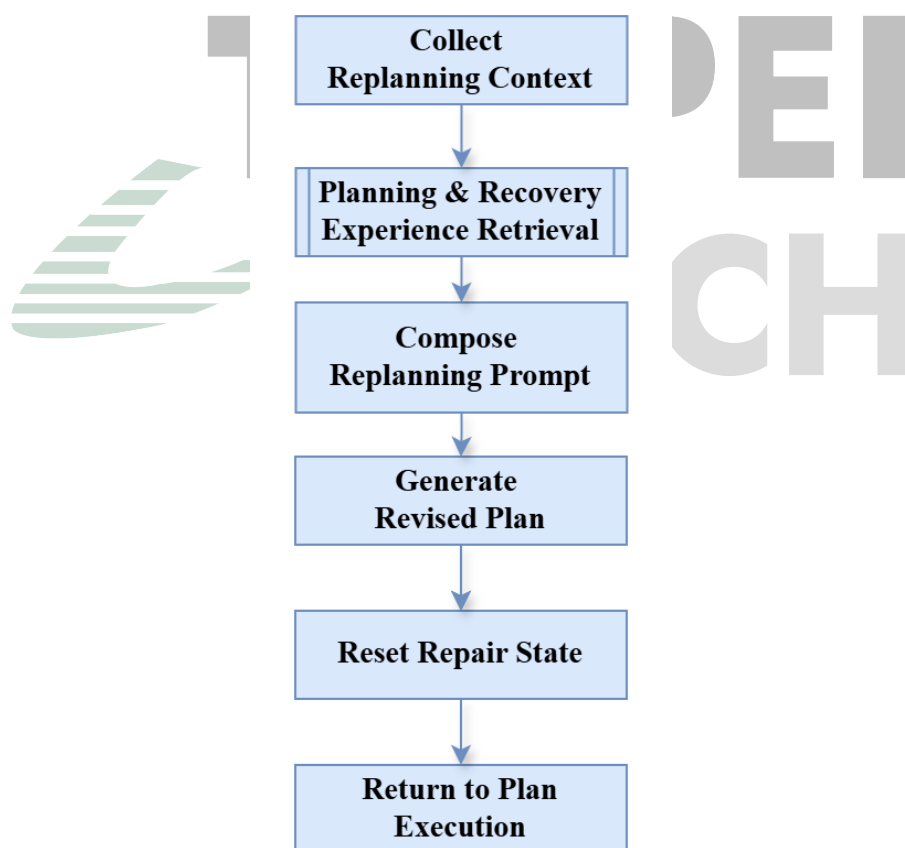


圖 4.4: 重新規劃提示組合與流程銜接圖

圖 4.4 呈現重新規劃階段如何結合原始任務、既有計畫、已完成結果、失敗脈絡與兩類經驗，形成重新規劃提示並回到任務執行流程。

此階段可整理為下列四個主要步驟：

- **收集重新規劃脈絡：**整理原始任務、既有計畫、已完成步驟結果與失敗資訊。
- **規劃與修復經驗檢索：**分別取得與任務結構及失敗情境相關的經驗。
- **重新規劃提示組合：**將任務脈絡、失敗資訊與兩類經驗組合為重新規劃提示。
- **產生後續計畫並回到執行：**產生修正後的後續計畫，重置修復相關暫存狀態，並交回步驟執行流程。

4.5.1 重新規劃觸發條件

重新規劃可由多個失敗處理分支觸發：

- **錯誤分類後直接回退：**若原始失敗步驟被暫定分類為不可行計畫，系統不進入局部修復，而是直接重新規劃。
- **修復限制檢查未通過：**若同一原始失敗點已達修復嘗試限制，或修復鏈深度已達上限，系統停止局部修復並重新規劃。
- **修復 API 搜尋或策略選擇失敗：**若系統無法取得可用候選修復 API，或模型輸出的策略無法對應到可套用的計畫更新，則回退至重新規劃。

若局部修復已成功更新目前計畫，系統會回到步驟執行流程；若上述任一條件使局部修復無法繼續，則進入重新規劃。

4.5.2 重新規劃提示組合與經驗注入

重新規劃階段會將任務脈絡、執行狀態、失敗資訊與兩類經驗組合為重新規劃提示。相較於初始規劃，重新規劃提示需要同時保留任務原始需求與已觀察到的失敗情境，使大型語言模型能在延續仍有效資料流程的前提下，重新產生後續計畫。

在提示內容上，重新規劃階段主要提供下列資訊：

- **任務與既有計畫：**包含原始任務目標與目前計畫，使重新規劃器能維持任務需求與已建立的計畫脈絡。

- **已完成步驟結果：**包含先前成功步驟留下的中間結果，作為後續計畫延續資料流程的依據。
- **失敗資訊：**包含目前失敗步驟、錯誤訊息與相關失敗脈絡，使重新規劃器能避開已觀察到的錯誤。
- **規劃導向經驗：**提供與任務結構、資料依賴與操作順序相關的策略參考。
- **修復導向經驗：**提供與目前失敗情境、常見陷阱與修復方向相關的策略參考。

當系統進入重新規劃時，會分別建立規劃導向查詢與修復導向查詢。規劃導向查詢著重於任務目標、既有計畫與資料流程，用於取得與任務結構相關的經驗；修復導向查詢則著重於失敗步驟、錯誤訊息與失敗脈絡，用於取得與失敗情境相關的經驗。兩類經驗會在提示中分別呈現，使重新規劃器能同時參考任務層級的規劃原則與錯誤情境下的修復提醒；經驗注入片段如附錄 A.4 所示。

4.5.3 重新規劃後的流程銜接

重新規劃節點的輸出為修正後的後續計畫。產生後續計畫後，系統會重置與局部修復相關的暫存狀態，並將新計畫交回任務執行流程。此設計使系統能於局部修復與重新規劃之間切換：局部修復先嘗試以小範圍計畫修改恢復執行；當局部修復無法成功更新目前計畫或達到限制時，重新規劃節點會重新產生後續任務流程。

第五章 實驗與結果分析

本章旨在評估 CBR-APIKGAgent 於多步驟 API 任務中的任務完成能力與錯誤修復表現。透過 AppWorld 任務環境進行實驗，觀察完整系統之整體表現，並進一步分析原子化經驗重用與動態局部修復機制各自對任務完成率造成的影響。

由於本研究的核心假設為「由訓練資料成功任務軌跡萃取之原子化經驗，可作為相似任務結構或錯誤情境下的策略參考」，以及「局部修復可在部分失敗情境下協助代理人由執行錯誤中恢復」，因此本章將分別從整體效能、消融實驗與案例分析等面向進行分析。實驗結果以實際執行紀錄與 AppWorld 評估結果為依據，並搭配可取得完整軌跡的案例，討論兩項機制如何在不同階段共同影響任務完成，以及完整系統目前呈現的限制。

5.1 研究問題

本研究設計四項研究問題（Research Questions, RQs），分別以 RQ1 至 RQ4 表示，以對應第一章所提出之研究目的：

1. RQ1：CBR-APIKGAgent 相較於既有方法在 AppWorld 任務中的整體表現為何？

此問題用於評估完整系統在 AppWorld test_normal 上的整體任務完成能力。

2. RQ2：原子化經驗重用機制對任務完成率造成何種影響？

此問題用於透過消融實驗分析原子化經驗重用機制對任務完成結果的影響。

3. RQ3：動態局部修復機制對任務完成率造成何種影響？

此問題用於透過消融實驗分析動態局部修復機制對任務完成結果的影響。

4. RQ4：原子化經驗與動態局部修復如何共同影響多步驟 API 任務的完成？

此問題延續消融實驗結果，透過可取得完整軌跡的成功與限制案例，觀察原子化經驗與動態局部修復分別在規劃與執行階段的作用，以及兩者如何共同影響任務完成。

5.2 實驗設置

5.2.1 資料集與任務範圍

本研究於 AppWorld [3] 環境中進行實驗。AppWorld 提供多應用程式 API 任務環境，任務完成與否由最終環境狀態是否符合預期條件判定。因此，代理人不僅需要產生合理文字回覆，也必須透過 API 呼叫與資料處理正確改變或查詢環境狀態。

正式實驗以 AppWorld test_normal 作為主要評估資料集，共包含 168 個任務，並以本研究提出之 CBR-APIKGAgent 作為評估系統。各項實驗所使用的任務範圍將於對應實驗小節中說明。

在每一項實驗設定中，各測試任務僅採用一次正式執行結果；局部修復、重新規劃與執行重試皆發生於該次任務執行期間，並受後述控制設定限制。任務結束後，AppWorld evaluation tests 的結果僅用於計算評估指標，不會回饋給代理人、寫入正式經驗庫，或作為調整後重新執行同一測試任務的依據。

5.2.2 評估指標

依據 AppWorld 官方評估方式，AppWorld 採用程式化且以最終狀態為基礎的 evaluation tests 判斷任務是否完成，並以任務目標完成率（Task Goal Completion, TGC）與情境目標完成率（Scenario Goal Completion, SGC）作為主要指標 [3]。

1. **任務目標完成率（TGC）**：TGC 表示通過所有 AppWorld evaluation tests 的任務比例。對於單一任務而言，只有當該任務的所有評估測試皆通過時，該任務才計為成功。其計算方式如下：

$$TGC = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\%$$

其中， N_{success} 代表通過所有 evaluation tests 的任務數量， N_{total} 代表評估任務總數。

2. **情境目標完成率（SGC）**：SGC 以 AppWorld 的 task scenario 為單位計算。只有同一 scenario 產生之所有任務皆通過 evaluation tests 時，該 scenario 才計為成功。此

指標用於衡量代理人是否能在同一任務情境的不同需求與初始狀態變化下穩定完成目標。其計算方式如下：

$$SGC = \frac{N_{\text{success scenarios}}}{N_{\text{total scenarios}}} \times 100\%$$

5.2.3 實驗環境與主要控制設定

本研究實驗之硬體與軟體環境如表 5.1 所示。由於本研究主要透過雲端大型語言模型服務進行推論，本地端硬體主要負責執行 AppWorld 環境、代理人控制流程、API 模擬環境與實驗紀錄整理。

表 5.1: 硬體與軟體環境

項目	設定
作業系統	Ubuntu 22.04.5 LTS (WSL2)
處理器	12th Gen Intel(R) Core(TM) i7-12700
記憶體	16 GB
Python 版本	Python 3.12
AppWorld 版本	0.1.3

除上述執行環境外，實驗亦需固定語言模型、推論溫度、執行預算、經驗檢索設定與局部修復限制等主要控制項。本研究採用 Gemini 2.5 Flash 作為大型語言模型 [25]；表 5.2 列出本研究實驗中需要固定或比較的重點設定。

5.3 實驗一：整體效能比較

本節對應 RQ1，目的在於呈現 CBR-APIKAgent 在 AppWorld test_normal 上的整體任務完成率，並與既有方法進行比較。本實驗使用完整 test_normal 168 個任務作為評估範圍，並依 AppWorld 官方評估結果計算 TGC 與 SGC。若任務執行後未產生完整評估報告或出現異常執行情形，該任務在統計中計為失敗。

為呈現本研究方法於 AppWorld 基準中的相對位置，表 5.3 列出既有方法於 test_normal 上的結果。比較對象包含 ReAct [18]，以及 AppWorld 所定義的 PlanExec、FullCodeRefl 與 IPFunCall 基線，其方法與數據取自 AppWorld 原論文 [3]；其餘方法包

表 5.2: 實驗主要控制設定

項目	設定
大型語言模型	gemini-2.5-flash
溫度參數	一般判斷：0；規劃與重新規劃：0.2；執行推論：0.2；API 候選評估：0.1
任務執行時間限制	3600 秒 / 題
執行流程內重新嘗試上限	3 次 / 題
局部修復上限	3 次 / 原始失敗步驟
修復鏈深度上限	2 個修復步驟 / 原始失敗步驟
經驗檢索相似度門檻	0.35
初始規劃經驗檢索	規劃導向經驗至多 3 筆
重新規劃經驗檢索	規劃導向經驗至多 1 筆；修復導向經驗至多 1 筆
局部修復經驗檢索	修復導向經驗至多 2 筆

括 LOOP [26]、IBM CUGA [27]、ReAct + 2 SetBSR Demos [28] 與 APIKGAgent [2]，相關數據主要取自各方法原論文與 AppWorld 官方 leaderboard [29]，其中 leaderboard 數據以 2026 年 7 月 13 日存取結果為準。LOOP 於表中採用官方 leaderboard 登載之 72.6% TGC 與 53.6% SGC；其原始論文則報告五次評估之平均結果 $71.3 \pm 1.3\%$ 與 $53.6 \pm 2.2\%$ ，兩者屬不同統計紀錄，本表以前者作為公開 leaderboard 比較值。由於不同方法可能採用不同模型、示範資料、訓練策略或執行預算，因此本表主要作為 benchmark 相對表現參考，而非嚴格控制變因下的直接比較。

由表 5.3 可知，CBR-APIKGAgent 於 test_normal 上達成 51.8% 的 TGC 與 35.7% 的 SGC。在表中採用 Gemini 2.5 Flash 的方法裡，本研究方法之兩項指標皆為最高，並高於同模型的 ReAct、FullCodeRefl 與 APIKGAgent；但與 IBM CUGA、LOOP 及 ReAct + 2 SetBSR Demos 等使用不同模型、示範資料或訓練策略的方法相比仍有差距。因此，針對 RQ1，本實驗顯示 CBR-APIKGAgent 改善了相同模型條件下的整體任務完成表現，但尚未達到 AppWorld 公開結果中的最佳水準。至於兩項核心機制各自的貢獻，將於下一節透過消融實驗進一步分析。

5.4 實驗二：消融實驗與案例分析

本節對應 RQ2、RQ3 與 RQ4。首先，透過消融實驗分別分析原子化經驗重用機制與動態局部修復機制對系統表現造成的影響，以回答 RQ2 與 RQ3。其次，本節延續消

表 5.3: 各方法於 AppWorld test_normal 之整體效能比較

模型	方法	TGC (%)	SGC (%)
GPT-4o	ReAct	48.8	32.1
	ReAct + 2 SetBSR Demos	68.5	57.1
	PlanExec	44.6	23.2
	FullCodeRefl	33.9	26.8
	IPFunCall	32.1	16.1
GPT-4.1	IBM CUGA	73.2	62.5
LLaMA3-70B	ReAct	20.8	8.9
	PlanExec	8.9	1.8
	FullCodeRefl	24.4	17.9
Qwen2.5-32B	ReAct	39.2	18.6
	LOOP	72.6	53.6
Gemini 2.5 Flash	ReAct	50.6	28.6
	FullCodeRefl	36.9	26.8
	APIKGAgent	41.7	17.9
	CBR-APIKGAgent	51.8	35.7

融實驗結果，透過可取得完整軌跡的成功與限制案例，觀察兩項機制如何在規劃與執行階段共同影響任務完成，以回答 RQ4。

5.4.1 實驗組別

消融實驗共包含四種設定，用以分別觀察原子化經驗與局部修復對任務完成結果的影響。四種設定皆於 AppWorld test_normal 的 168 個任務進行評估，並採相同計分方式。除上述機制差異外，其餘模型、時間限制、重試上限與評估方式皆保持一致。

5.4.2 消融實驗結果

正式消融實驗結果如表 5.4 所示。四種設定皆使用相同任務集合，並以 TGC 與 SGC 作為比較指標。

表 5.4 將用於比較原子化經驗、局部修復與完整系統設定之 TGC 與 SGC 差異。本節後續將先分析原子化經驗與局部修復各自的影響，再透過完整系統與可取得完整軌跡的案例，討論兩項機制如何在規劃與執行階段共同影響任務完成。

表 5.4: 消融實驗結果

設定	TGC (%)	SGC (%)
完整系統	51.8	35.7
僅局部修復	49.4	28.6
僅原子化經驗	43.5	25.0
基礎系統	37.5	14.3

5.4.3 單一機制效益分析

本節分別比較僅原子化經驗與僅局部修復兩種設定相較於基礎系統的差異，以回答 RQ2 與 RQ3。

原子化經驗之影響

由表 5.4 可知，基礎系統之 TGC 為 37.5%，SGC 為 14.3%。僅原子化經驗設定之 TGC 為 43.5%，SGC 為 25.0%，相較於基礎系統分別提升 6.0 與 10.7 個百分點。此結果顯示，在未啟用動態局部修復的情況下，原子化經驗本身仍可改善任務層級與情境層級的完成率。

進一步比對「基礎系統失敗、僅原子化經驗成功」的任務，並檢視其中可取得完整軌跡的案例後可觀察到，原子化經驗主要在初始規劃或重新規劃階段提供可重用的任務策略，例如完整資料取得、篩選條件設定與聚合前檢查等。換言之，原子化經驗主要協助代理人建立較完整的任務處理流程，而非直接修復執行階段的單一步驟錯誤。

範例：

- **案例任務：**使用者需計算過去 5 天內 Venmo 所收到的付款請求總金額。
- **消融差異：**此任務在基礎系統中未通過評估，但在僅原子化經驗設定中成功。
- **基礎系統失敗原因：**基礎系統雖能呼叫相關 API，但在資料取得完整性、日期篩選條件套用或聚合結果正確性上仍有不足，因此最終結果未通過評估。
- **機制作用：**僅原子化經驗設定在此任務中通過評估；從可檢視的同類軌跡可觀察到，相關經驗會引導系統納入完整資料取得、分頁處理與聚合前篩選等策略，使系統較能形成「先完整取得資料，再依條件篩選並統計」的處理流程。

- **影響：**此例顯示，原子化經驗可協助系統將「先完整取得資料，再篩選與統計」的可重用策略納入計畫，改善查詢與聚合任務中的資料依賴設計，而非僅提升單一步驟的 API 呼叫成功率。

動態局部修復之影響

僅局部修復設定之 TGC 為 49.4%，SGC 為 28.6%，相較於基礎系統分別提升 11.9 與 14.3 個百分點。此結果顯示，在不加入原子化經驗的情況下，僅透過執行失敗後的局部計畫修正，仍可使部分原本失敗的任務恢復並完成。進一步檢視「基礎系統失敗、僅局部修復成功」中可取得完整軌跡的案例可觀察到，這些成功案例較能排除由經驗檢索造成的規劃差異；主要差異出現在執行失敗後，系統可根據錯誤訊息、可行回饋、候選 API 與當前相依資料，在原計畫附近插入或替換局部步驟。

從已檢視的案例來看，局部修復可處理三類執行過程中的問題。第一類是缺少必要的前置資訊或授權，例如原先的操作需要 access token，卻因授權失敗或缺少必要資訊而無法執行；此時系統會補入登入步驟，使後續查詢或操作得以繼續。第二類是原先選用的功能不適合或無法使用，例如嘗試取得不存在的 incoming request；此時系統會改用可實際執行的功能，或補入必要步驟。第三類是取得的資料不足，或查詢條件不夠精確，例如聯絡人關係設定過於寬泛或不正確，導致後續無法取得所需資訊；此時系統可依據失敗回饋補入較合適的查詢步驟。因此，就已檢視的案例而言，動態局部修復的作用主要在於執行過程中利用局部錯誤證據修正當前失敗點。其效果可見於授權缺失、必要前置資料缺失、API 選擇錯誤與局部資料取得不完整等情境，使系統有機會在不重新規劃整個任務的情況下恢復執行。

範例：

- **案例任務：**使用者需更新 Spotify 播放清單，依據兄弟姐妹在手機訊息中的建議調整歌曲內容。
- **消融差異：**此任務在基礎系統中失敗，但在僅局部修復設定中成功。
- **基礎系統失敗原因：**初始失敗原因是代理人先用 relationship = “family” 查詢聯絡人，但結果為空，因而無法取得兄弟姐妹的電話號碼。

- **機制作用：**系統根據錯誤回饋在原計畫附近插入新的聯絡人查詢步驟，改用 relationship = “sibling” 重新查詢，取得正確聯絡人後再繼續執行後續任務。
- **影響：**此例呈現局部修復可透過插入單一步驟，恢復執行流程並避免整體重新規劃。

整體而言，原子化經驗與動態局部修復在單獨啟用時皆高於基礎系統。針對 RQ2，僅原子化經驗設定使 TGC 與 SGC 分別提升 6.0 與 10.7 個百分點，顯示其主要透過提供資料取得、條件篩選與操作順序等策略參考改善規劃完整性。針對 RQ3，僅局部修復設定使 TGC 與 SGC 分別提升 11.9 與 14.3 個百分點，顯示其可利用執行階段的局部錯誤證據修正失敗步驟，提升任務恢復能力。在本組實驗中，僅局部修復設定之兩項指標皆高於僅原子化經驗設定，但兩項機制分別作用於規劃與執行恢復階段，對任務完成結果產生不同層次的影響。

5.4.4 完整系統與代表性案例分析

本節對應 RQ4，目的在於說明原子化經驗與動態局部修復如何共同影響任務完成。相較於前一節以消融數據分析兩項機制各自對任務完成率的影響，本節先透過可取得完整軌跡的成功案例，觀察兩者在規劃與執行階段的互補作用；再透過限制案例，討論目前完整系統仍可能面臨的限制。

完整系統同時啟用原子化經驗與動態局部修復，其 TGC 為 51.8%，SGC 為 35.7%，為四種消融設定中最高。相較於基礎系統，完整系統之 TGC 提升 14.3 個百分點，SGC 提升 21.4 個百分點；相較於僅原子化經驗設定，完整系統之 TGC 與 SGC 分別提升 8.3 與 10.7 個百分點；相較於僅局部修復設定，完整系統之 TGC 與 SGC 分別提升 2.4 與 7.1 個百分點。

互補案例分析

根據可取得完整軌跡的完整系統成功案例可觀察到，原子化經驗可在規劃階段提供目標選擇與處理策略的參考；若執行過程仍出現授權或前置資訊缺失，動態局部修復則可依據當下錯誤補足必要步驟。此類案例顯示，兩項機制作用於不同階段，並可能同一流程中共同影響任務完成。

範例：

- **案例任務：**使用者需將手機聯絡人中的 coworkers 與 friends 加入 Venmo friends，但僅限於目前尚未是 Venmo friends 的對象。
- **消融差異：**此任務在基礎系統中失敗，但在完整系統中成功。
- **基礎系統失敗原因：**基礎系統未能在操作前完整辨識 coworkers 與 friends，並排除既有的 Venmo friends，導致新增目標集合不完整或包含不應加入的對象，因而未能完成任務。
- **機制作用：**原子化經驗協助系統在規劃階段先建立尚未加入者的目標集合；執行過程出現 Venmo authentication error 時，動態局部修復再補入登入步驟，使後續查詢與新增好友操作得以繼續。
- **影響：**此例顯示，原子化經驗主要改善目標選擇與篩選，動態局部修復則負責執行階段的錯誤恢復；兩者結合後可同時處理目標集合定義與執行錯誤。

完整系統的限制分析

雖然完整系統在整體 TGC 與 SGC 上皆為四種設定中最高，但在個別任務中，並不保證一定優於僅啟用單一機制的設定。這類情況不宜直接解讀為原子化經驗與動態局部修復彼此干擾；較合理的解釋是，完整系統的效果仍會受到任務結構、搜尋式 API 語意，以及後續目標集合維持方式影響。特別是在涉及「先搜尋候選結果，再執行寫入操作」的任務中，若系統未能持續保留原始任務條件與候選結果之間的對應關係，後續局部修復即使能處理授權或缺少查詢步驟等執行錯誤，也未必能修正已被擴大的操作目標集合。

以下案例呈現其中一種失敗型態，並非所有完整系統失敗案例的共同原因。

範例：

- **案例任務：**使用者需將 Venmo 上的朋友名單調整為與手機聯絡人中的朋友一致，必要時執行新增與移除操作。
- **消融差異：**此任務中，基礎系統與僅局部修復設定皆成功，但完整系統失敗。

- **完整系統失敗原因：**完整系統雖在初始計畫中納入手機聯絡人中 friend 關係的篩選，但後續使用 venmo.search_users 將 phone friends 對應到 Venmo users 時，未充分處理搜尋 API 可能回傳廣泛候選結果的情況。系統將搜尋結果視為可直接操作的目標集合，而沒有再次依據原始 phone friend emails 驗證候選使用者身分，導致新增目標集合被擴大，最終好友新增與移除結果未符合評估條件。
- **局部修復限制：**局部修復可處理登入、授權或缺少查詢步驟等執行階段錯誤；但當上游已產生錯誤或不穩定的目標集合時，修復雖可補入查詢或重試操作，仍未必能回復原本應被保留的任務限制。
- **影響：**此例顯示，完整系統雖能結合經驗檢索與局部修復，但在涉及搜尋式 API 與寫入操作的任務中，仍需要明確維持「原始任務條件 → 候選結果 → 實際操作目標」之間的對應關係。若搜尋結果被誤視為精確匹配，後續局部修復即使能補足授權或查詢步驟，也未必能避免錯誤目標被寫入。

因此，針對 RQ4，消融結果與代表性案例顯示，原子化經驗主要在規劃階段提供任務結構與資料流策略，動態局部修復則在執行失敗後調整局部計畫，兩者因作用於不同階段而形成互補，使完整系統取得四種設定中的最佳整體結果。然而，兩項機制的結合並非在所有任務中皆能帶來正向效果；當檢索經驗未能維持任務限制，或上游目標集合已產生偏差時，局部修復未必能恢復正確的任務條件。此結果指出未來可從三個方向持續改善：在經驗使用時納入任務限制、細化局部修復與重新規劃的適用範圍，以及建立更可靠的經驗累積機制，以維持任務條件、候選結果與實際操作目標之間的對應關係。

第六章 結論與未來研究方向

6.1 結論

本研究針對大型語言模型代理人在多步驟 API 任務中所面臨之規劃經驗未能有效重用，以及執行失敗後缺乏局部修復能力等問題，提出案例式修復 API 知識圖譜代理人（Case-Based Repair API Knowledge Graph Agent, CBR-APIKGAgent）。本研究以 APIKGAgent 為基礎，設計原子化經驗重用機制與動態局部修復機制，使代理人在初始規劃、重新規劃與局部修復階段能參考由訓練資料成功任務軌跡萃取而來的策略經驗，並在步驟執行失敗時根據當前錯誤訊息、相依資料與執行結果，嘗試以插入、替換或刪除步驟的方式修正局部計畫。

實驗結果顯示，CBR-APIKGAgent 在 AppWorld test_normal 上較基礎系統具有更好的任務完成能力。消融實驗進一步顯示，原子化經驗與動態局部修復皆與任務完成率提升相關；從案例觀察，原子化經驗主要在規劃階段提供資料依賴與目標選擇的策略參考，動態局部修復則主要支援執行階段的錯誤恢復。兩項機制結合後取得最佳整體表現，顯示其在不同階段的作用可能共同影響任務完成。

從任務軌跡分析可觀察到，原子化經驗主要有助於代理人補足多步驟任務中的中間概念、資料依賴與寫入前檢查，例如完整資料取得、分頁處理、目標集合篩選與差集計算等。動態局部修復則較常處理執行階段暴露出的局部錯誤，例如授權資訊缺失、必要查詢步驟不足、前置資料不完整，或單一步驟操作對象不完整等情況。完整系統的案例分析顯示，兩項機制可在同一任務流程中分別作用於規劃與執行階段；然而，單一任務的結果仍會受到檢索而得的經驗是否符合任務限制、搜尋式 API 的回傳行為、初始規劃品質，以及系統對局部修復或重新規劃的選擇影響。因此，本研究的實驗結果支持經驗重用與局部修復對多步驟 API 任務具有實際助益，同時也說明未來仍需要更細緻地控制經驗使用、修復範圍與經驗累積的可靠性。

整體而言，本研究的主要貢獻包含三點。第一，將訓練資料中成功任務軌跡萃取為可跨任務檢索的原子化經驗，並將其應用於規劃與修復脈絡中。第二，設計動態局部修復流程，使代理人在步驟失敗後不必立即完全依賴重新規劃，而能先根據失敗資訊與相依資料嘗試局部調整。第三，透過整體效能比較、消融實驗與案例分析，評估

原子化經驗與動態局部修復如何在規劃與執行階段共同影響任務完成，並整理兩項機制在實際任務中的作用條件與限制。

6.2 未來研究方向

本研究雖已初步驗證原子化經驗重用與動態局部修復在多步驟 API 任務中的效果，仍有數個方向可進一步探討。

任務限制導向的經驗使用

原子化經驗不應只根據語意相似度被檢索與使用，也需要符合當前任務的限制、目標集合與操作條件。未來可進一步加入任務限制比對或約束檢查機制，讓系統在選擇經驗時，同時考慮該經驗是否對應到目前任務的關鍵條件，例如正確的資料來源、目標範圍與 API 層級操作限制。如此可降低「語意相近但任務不適用」的經驗被過度使用。

修復範圍細化與重新規劃切換

本研究已具備初步的局部修復與重新規劃切換機制，例如依據修復次數、修復深度與錯誤類型決定後續流程。然而，若初始規劃已建立錯誤的目標集合、資料依賴或計畫結構，持續進行局部修復未必能恢復任務。未來可進一步細化判斷機制，讓系統更準確地辨識哪些錯誤適合局部修復，哪些情況應盡早轉為重新規劃，以避免在不適合修補的計畫上浪費修復預算。

修復經驗可靠性評估

若系統未來希望持續從修復軌跡中累積新經驗，便不能只因為某個修復步驟或 API 呼叫成功就直接納入經驗庫。未來需要建立更可靠的經驗品質評估機制，根據執行證據、狀態變化、一致性檢查、後續步驟穩定性或其他可觀察訊號，評估 repair experience 是否具有重用價值，並避免把偶然有效但不穩定的修復策略誤存為長期經驗。

參考文獻

- [1] L. Wang et al., “A survey on large language model based autonomous agents,” *Front. Comput. Sci.*, vol. 18, no. 6, 2024, Art. no. 186345. DOI: 10.1007/s11704-024-40231-1.
- [2] 吳柏諭，一個由 LLM 驅動之 AI 代理人透過操作 API 自動完成任務之研究，碩士論文，國立臺北科技大學資訊工程系，2025。
- [3] H. Trivedi et al., “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 16022–16076. DOI: 10.18653/v1/2024.acl-long.850.
- [4] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 68539–68551.
- [5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” in *Advances in Neural Information Processing Systems*, vol. 37, Curran Associates, Inc., 2024, pp. 126544–126565. DOI: 10.52202/079017-4020.
- [6] Y. Qin et al., “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *International Conference on Learning Representations*, 2024.
- [7] Y. Du, F. Wei, and H. Zhang, “AnyTool: Self-reflective, hierarchical agents for large-scale API calls,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, PMLR, Jul. 2024, pp. 11812–11829.
- [8] Y. Song et al., “RestGPT: Connecting large language models with real-world RESTful APIs,” *arXiv preprint arXiv:2306.06624*, Jun. 2023. DOI: 10.48550/arXiv.2306.06624.
- [9] A. Aamodt and E. Plaza, “Case-based reasoning: Foundational issues, methodological variations, and system approaches,” *AI Commun.*, vol. 7, no. 1, pp. 39–59, 1994. DOI: 10.3233/AIC-1994-7104.
- [10] K. Wilkerson and D. Leake, “On implementing case-based reasoning with large language models,” in *Case-Based Reasoning Research and Development*, ser. Lecture Notes in Computer Science, vol. 14775, Springer Nature Switzerland, 2024, pp. 404–417. DOI: 10.1007/978-3-031-63646-2_26.

- [11] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, Association for Computing Machinery, Oct. 2023, pp. 1–22. DOI: 10.1145/3586183.3606763.
- [12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 8634–8652.
- [13] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, pp. 19632–19642, Mar. 2024. DOI: 10.1609/aaai.v38i17.29936.
- [14] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, PMLR, Jul. 2025, pp. 63897–63911.
- [15] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.
- [16] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474.
- [17] L. Wang et al., "Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 2609–2634. DOI: 10.18653/v1/2023.acl-long.147.
- [18] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.
- [19] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 46534–46594.
- [20] X. Zhang et al., "Divide-Verify-Refine: Can LLMs self-align with complex instructions?" in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 13783–13800. DOI: 10.18653/v1/2025.findings-acl.709.
- [21] K. Zhu et al., "Where LLM agents fail and how they can learn from failures," *arXiv preprint arXiv:2509.25370*, Sep. 2025. DOI: 10.48550/arXiv.2509.25370.

- [22] M. Fox, A. Gerevini, D. Long, and I. Serina, “Plan stability: Replanning versus plan repair,” in *Proceedings of the 16th International Conference on Automated Planning and Scheduling*, AAAI Press, 2006, pp. 212–221.
- [23] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, “AdaPlanner: Adaptive planning from feedback with language models,” in *Advances in Neural Information Processing Systems*, vol. 36, Curran Associates, Inc., 2023, pp. 58202–58245.
- [24] A. Prasad et al., “ADaPT: As-needed decomposition and planning with language models,” in *Findings of the Association for Computational Linguistics: NAACL 2024*, Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 4226–4252. DOI: 10.18653/v1/2024.findings-naacl.264.
- [25] Gemini Team, “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities,” *arXiv preprint arXiv:2507.06261*, Jul. 2025. DOI: 10.48550/arXiv.2507.06261.
- [26] K. Chen et al., “Reinforcement learning for long-horizon interactive LLM agents,” *arXiv preprint arXiv:2502.01600*, Feb. 2025. DOI: 10.48550/arXiv.2502.01600.
- [27] S. Marreed et al., “Towards enterprise-ready computer using generalist agent,” *arXiv preprint arXiv:2503.01861*, Mar. 2025. DOI: 10.48550/arXiv.2503.01861.
- [28] S. Gupta, S. Singh, A. Sabharwal, T. Khot, and B. Bogin, “Leveraging in-context learning for language model agents,” *arXiv preprint arXiv:2506.13109*, Jun. 2025. DOI: 10.48550/arXiv.2506.13109.
- [29] AppWorld. “AppWorld Leaderboard,” [Online; accessed 13-July-2026]. Available: <https://appworld.dev/appworld>

附錄 A 核心提示詞、輸入資料與輸出格式

A.1 原子化經驗萃取 Prompt 與輸出範例

System Prompt

You are an AI agent methodology researcher. Your job is to distill train-dataset successful trajectories into reusable ATOMIC experiences.

An atomic experience is NOT a full task recipe and NOT a skill. It should capture a reusable data-flow, target-set, constraint, or write-safety principle that can help future planners, retry planners, or recovery steps.

Generate 0-2 high-quality atomic experiences from the evidence. Prefer 1. Generate 2 only if there are two clearly distinct reusable patterns. Prefer no experience over a task-specific or low-value one.

Important constraints:

- Do NOT summarize the full task workflow.
- Do NOT produce one experience just because one task was shown.
- Do NOT create a task-specific recipe like 'first call A, then call B'.
- A good experience must name a reusable data-flow risk, the intermediate concepts needed to control it, and the read/write behavior it protects.
- Reject generic engineering advice such as rate limits, concurrency, authentication, or pagination unless it is central to task correctness in the evidence.
- Avoid full API call examples. Examples should illustrate data shapes, stable identifiers, or concept meaning.
- Focus on transferable principles: target set provenance, batch completeness, process output contracts, write target guards, state comparison before writes, search/lookup result usage, or preserving existing content format.

experience_type is a soft label. Prefer one of these broad labels when it fits: target_set_provenance, batch_completeness, process_output_contract, write_target_guard. If none fits, create a concise snake_case label.

Keep each experience concise:

- when_to_use: 1 sentence.
- core_principle: 1 sentence.
- required_intermediate_concepts: list only genuinely necessary concepts.
- required_intermediate_concepts.name: lowercase snake_case only.
- required_intermediate_concepts must be data-flow concepts the agent must preserve, name, pass forward, or check. They are NOT API names, operation names, tool names, or execution steps.

- how_to_apply: 2-4 short bullets.
- pitfalls: 2-4 short bullets.

Return JSON with an 'experiences' array. Each experience must have exactly these keys: experience_type, source_task_id, source_task_goal, when_to_use, core_principle, required_intermediate_concepts, how_to_apply, pitfalls.

User Prompt 模板

Original Train Task:

```
{task_instruction}
```

Evidence Package:

specs.json:

```
{specs_json}
```

public_data.json:

```
{public_data_json}
```

solution.py:

```
{solution_py}
```

evaluation.py:

```
{evaluation_py}
```

api_calls.json compacted (raw={api_calls_count}, compacted={api_calls_compacted_count}):

```
{api_calls_compacted}
```

Distill only reusable atomic experiences. If the evidence only supports a task-specific recipe, return an empty experiences list.

輸出 JSON Schema

```
{
  "experiences": [
    {
      "experience_type": "string",
      "source_task_id": "string",
      "source_task_goal": "string",
      "when_to_use": "string",
      "core_principle": "string",

```

```

    "required_intermediate_concepts": [
      {
        "name": "string",
        "role": "string",
        "example": "string"
      }
    ],
    "how_to_apply": ["string"],
    "pitfalls": ["string"]
  }
]
}

```

實際輸出範例

```

{
  "experience_type": "target_set_provenance",
  "source_task_id": "82e2fac_2",
  "source_task_goal": "What is the title of the least-played song in my Spotify song library.",
  "when_to_use": "When collecting identifiers for a target set from multiple sources or paginated results that might contain duplicates.",
  "core_principle": "Ensure the final set of identifiers used for subsequent operations is complete and unique, preventing redundant calls and ensuring all relevant data is processed.",
  "required_intermediate_concepts": [
    {
      "name": "paginated_api_results",
      "role": "The raw, potentially overlapping or incomplete list of items returned from paginated API calls.",
      "example": "[{\"song_id\": \"s1\", \"name\": \"Song A\"}, {\"song_id\": \"s2\", \"name\": \"Song B\"}, {\"song_id\": \"s1\", \"name\": \"Song A\"}]"
    },
    {
      "name": "extracted_identifiers",
      "role": "A list of identifiers extracted from all items across all paginated results.",
      "example": "[\"s1\", \"s2\", \"s1\"]"
    },
    {
      "name": "unique_target_identifiers",
      "role": "The final, deduplicated set of identifiers to be used for detailed fetching or processing.",
      "example": "[\"s1\", \"s2\"]"
    }
  ],
}

```

```

"how_to_apply": [
  "Use a robust pagination utility to ensure all data pages are retrieved.",
  "Extract a stable identifier from each item across all retrieved pages.",
  "Aggregate all extracted identifiers into a unique collection.",
  "Use this unique set of identifiers for subsequent API calls to fetch detailed information."
],
"pitfalls": [
  "Failing to handle pagination, leading to an incomplete target set.",
  "Not deduplicating identifiers, resulting in redundant API calls.",
  "Extracting an unstable or non-unique identifier.",
  "Assuming all items from different sources are distinct without explicit deduplication."
]
}

```

A.2 錯誤分類與可行回饋 Prompt

System Prompt

```

You are an advanced error analyst for an autonomous agent.
Your task is to analyze the provided error message, the current step instruction, and the failed API
information to:
1. Classify the error into exactly one of the provided error categories.
2. Generate specific, actionable feedback (repair instruction) to resolve the error.

# Advanced Diagnostics (Rich Context)
If 'Error Context' is provided, use it to perform deep diagnostics:
- Missing Parameters: check 'current_memory_keys' in the context. If a missing parameter has a
  potential match in memory, explicitly suggest using that specific key in your feedback.
- Candidate APIs: If 'suggested_apis' are present, verify if they align with the goal and suggest
  using them in your feedback.
- Execution Errors:
  - For any execution error, cross-reference the details with the params_used and the api_doc_snippet.
  - Identify specifically if a parameter's type or value in params_used violates the requirements
    defined in api_doc_snippet.
  - Look for discrepancies between the error message and the actual parameters provided.

# Error Taxonomy
{error_taxonomy}

# Output Format
Return a JSON object:
{

```

```
"category": "<one of the keys from taxonomy>",  
"feedback": "<specific actionable instruction to fix this error>"  
}
```

User Prompt 模板

```
Current Step Instruction: {current_step_instruction}  
Failed API Info: {failed_api_info}  
Error Message: {error_message}  
Error Context (Rich Info): {error_context}
```

錯誤類型範圍

```
memory: oversimplification, false_memory, retrieval_failure  
reflection: progress_misassessment, outcome_misinterpretation, wrong_causal_attribution,  
           reflection_hallucination  
planning: constraint_ignorance, impossible_plan, inefficient_plan  
action: plan_misalignment, format_error, parameter_error  
system: step_limit_reached, tool_execution_error, llm_limitation, environment_error  
verification/auth: auth_error, unknown
```

輸出 JSON Schema

```
{  
  "category": "string",  
  "feedback": "string"  
}
```

A.3 局部修復策略選擇 Prompt

System Prompt 摘要

```
You are choosing ONE best API (or action) to attempt recovery from a failed step in a multi-step plan.  
  
You must:  
1. Read the originally failed step's objective, the failure message, and the intended repair action.  
2. Examine each candidate API.  
3. Pick the single most relevant API.
```

```

- If none are helpful, return chosen = "NONE".
4. Determine keep_dependency for REPLACE or INSERT modes.
5. Choose the plan edit mode and repair intent.

Repair intent:
- MISSING_PREREQUISITE:
    The selected API obtains missing prerequisite information or state needed before retrying the failed
    step, such as login, search, lookup, get, list, retrieve, or fetch.

- COMPLETE_PARTIAL_EXECUTION:
    The failed step is conceptually the right operation, but its parameters, target coverage, or
    repeated execution were incomplete.

- CORRECT_WRONG_OPERATION:
    The failed step uses the wrong API, wrong step type, wrong target, or wrong operation.

- REMOVE_REDUNDANT_STEP:
    The failed step is unnecessary, duplicate, harmful, or contradicted by new evidence.

Mode decision rules:
- For MISSING_PREREQUISITE, choose INSERT.
- For COMPLETE_PARTIAL_EXECUTION, choose REPLACE.
- For CORRECT_WRONG_OPERATION, choose REPLACE.
- For REMOVE_REDUNDANT_STEP, choose DELETE.
- Do not choose INSERT when the selected API and the failed step have the same operational goal;
  choose REPLACE instead.

```

User Prompt 模板

```

Originally Failed Step: {failed_step_text}
Failure Message: {failure_msg}
Repair Action: {actionable_feedback}
Candidates: {candidate_apis}

Failure Evidence:
{
  "error_code": "...",
  "details": "...",
  "api_name": "...",
  "analysis_reason": "...",
  "dependency_context": "...",
  "call_experience": "..."
}

```



```
Historical Recovery Strategy:
- Proven Recovery Strategy 1:
  {retrieved_recovery_experience}
- Proven Recovery Strategy 2:
  {retrieved_recovery_experience}
```

輸出 JSON Schema

```
{
  "chosen": "string",
  "keep_dependency": true,
  "mode": "INSERT | REPLACE | DELETE",
  "repair_intent": "MISSING_PREREQUISITE | COMPLETE_PARTIAL_EXECUTION | CORRECT_WRONG_OPERATION |
  REMOVE_REDUNDANT_STEP"
}
```

A.4 規劃與重新規劃的經驗注入片段

Initial Planning 經驗注入

```
# Lessons Learned from Previous Tasks
```

The following are retrieved experiences. Use them as compact guidance for clearer step context and safer data flow; do not copy them as task-specific recipes.

If a lesson uses words such as set, collection, tuple, or object, treat them as semantic data-flow concepts unless it explicitly says otherwise. Process step final outputs must still be JSON-like dictionaries/lists, such as a list of unique IDs under a meaningful key, not Python set/tuple objects.

```
- Strategic Lesson 1:
{retrieved_atomic_experience}
```

```
- Strategic Lesson 2:
{retrieved_atomic_experience}
```

```
- Strategic Lesson 3:
{retrieved_atomic_experience}
```

Replanning 共用引導

```
# Case-Based Replan Guidance
```

Use these retrieved lessons as hints when revising the failed plan. Planning lessons help restructure the workflow; recovery lessons help address the specific failure.

If a lesson uses words such as set, collection, tuple, or object, treat them as semantic data-flow concepts unless it explicitly says otherwise. Process step final outputs must still be JSON-like dictionaries/lists, such as a list of unique IDs under a meaningful key, not Python set/tuple objects.

Planning / Recovery Lessons 注入區塊

```
## Planning Lessons
```

```
- Planning Lesson 1 [id=P1] (source={source}, similarity={score}):  
{retrieved_planning_lesson}
```

```
## Recovery Lessons
```

```
- Recovery Lesson 1 [id=R1] (source={source}, similarity={score}):  
{retrieved_recovery_lesson}
```